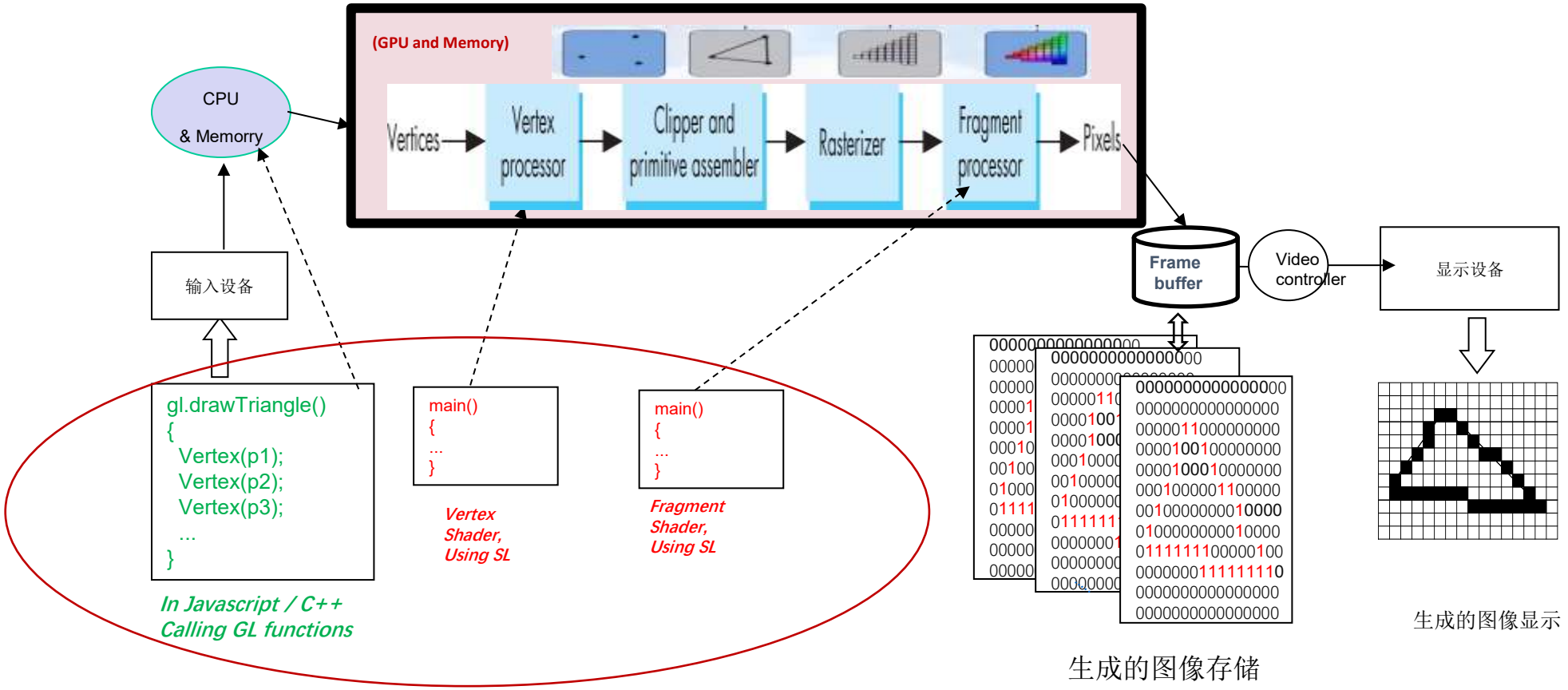
Recap

The Programmable Rendering Pipeline 可编程渲染管线的架构

- Performance is achieved by using GPU rather than CPU,GPU does all rendering,
- Programmer Control GPU through shaders
  - Application's job (run at CPU) is to send data to GPU
  - Vertext Shader run at per vertex, fragment shader run at per fragment concurrently

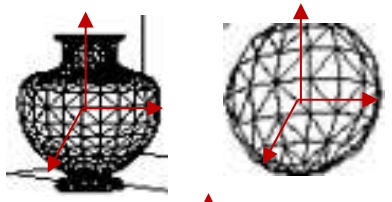


Modeling/Geometry

Room?

Geometry modeling几何造型

Objects and Material Representation Data,



MC

大量的初始数据

Interaction and Animation 交互和动画

SC/DC



colored Image 2D

少量变化的参数

VC

Rendering渲染

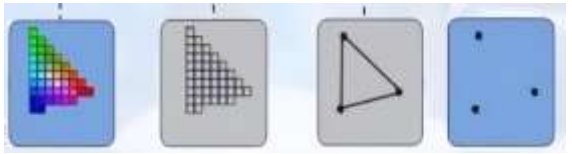
~GPU完成~

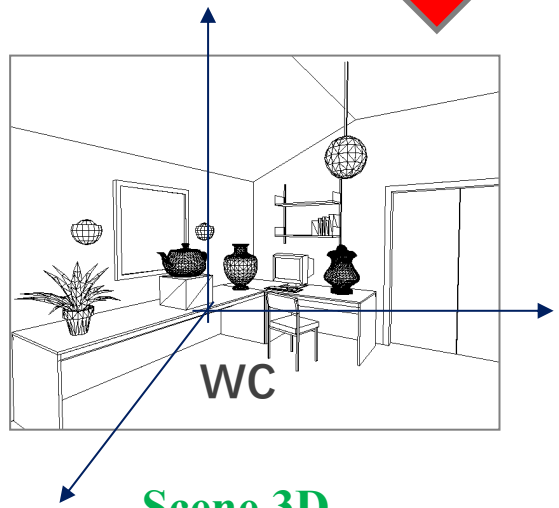
片元处理: Fragment

光栅化 Rasterization

图元组装和裁剪clip

顶点处理: Vertex





WC

Scene 3D

Scene description: objects, light etc.

Today Topics

- **1.了解"几何表示"方法:图形对象的数字化表示方法**
- **2. 了解“造型技术”，主要分实体建模，过程建模两大类**
- **3. 掌握 网格mesh, 参数曲线曲面spline，分形建模方法，作为后续场景中物体表示方法**

➢参考:

1.万琳　大学MOOC

2.B站: GAMES101 lec10 ~lec12 Gemometry

➢继续学习:

1.刘利刚 :https://faculty.ustc.edu.cn/liuligang/zh_CN/index.htm

B站课件:GAMES 102课程《几何建模与处理基础》，GAMES 301课程《曲面参数化》)

2.体表示和体渲染

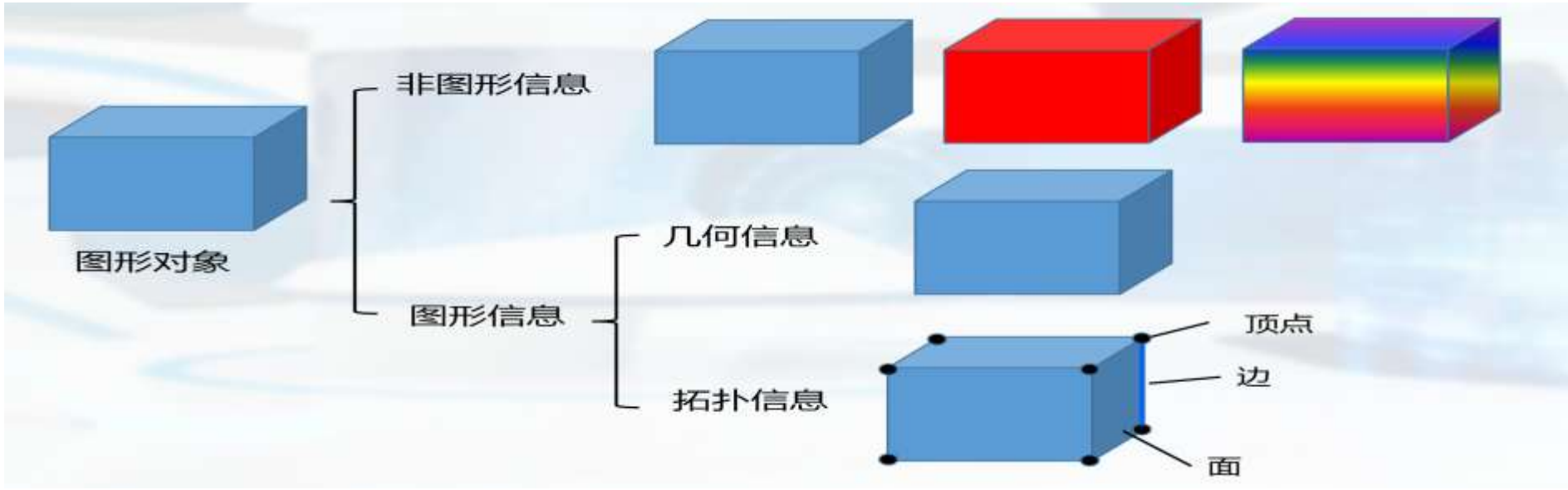
Outline

- Properties of Geometric Objects几何对象的属性
- Geometry Objects Representation几何对象的表示
- Modeling Technology造型技术
  - 实体造型Solid Modeling*
    - CSG(Constructive Solid Geometry ）构造实体几何
    - SP(Space-Partitioning Representation）空间分割表示
    - B-reps(Boundary Representation)边界表示*
      - Mesh网格*
      - Spline Curve and Surface 样条/*
  - 过程建模Process Modeling
    - Fractal geometry分形几何
    - Particle system粒子系统

Properties of Geometric Objects

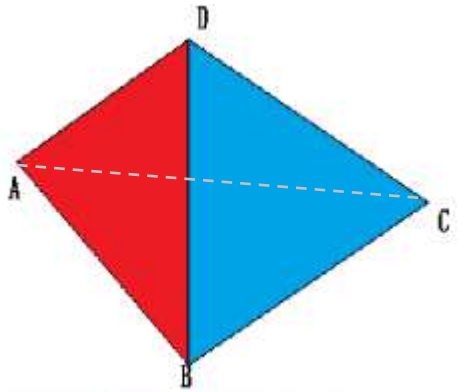
➢图形对象的属性

- **非图形信息**：**线型，颜色，亮度，质量，比重，体积等。**
- **图形信息**：**对象和构成它的点、线、面的位置和相互关系，几何尺寸等**（造型关注）
  - ➢**几何信息**：形体在欧氏空间中的位置和大小。
  - ➢拓扑信息：形体各分量的数目和相互连接关系。

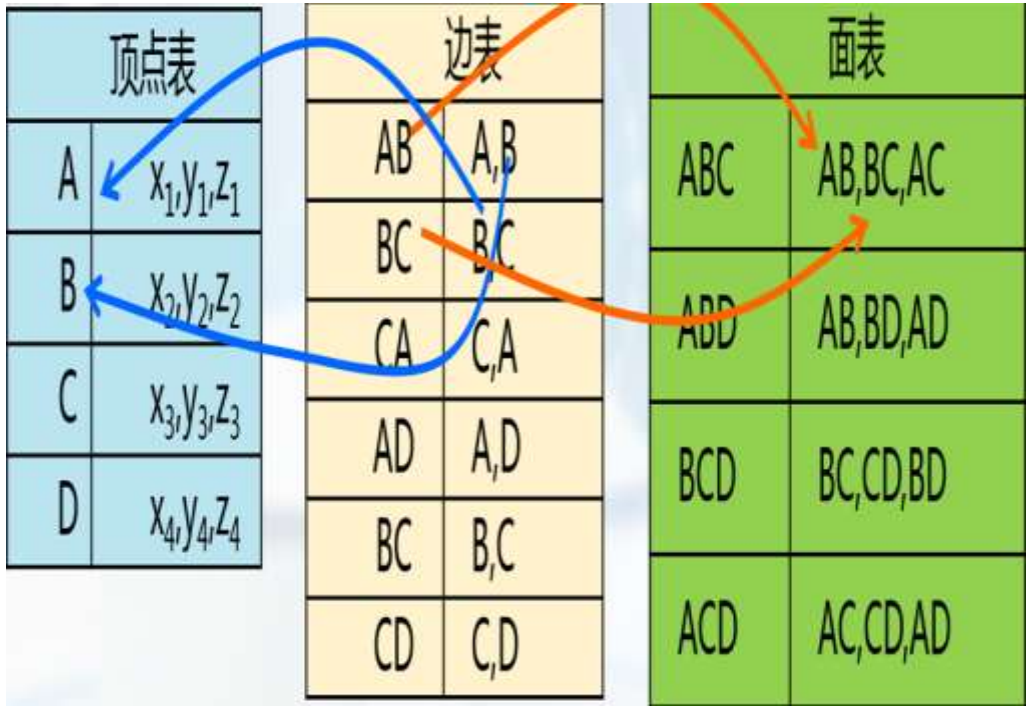


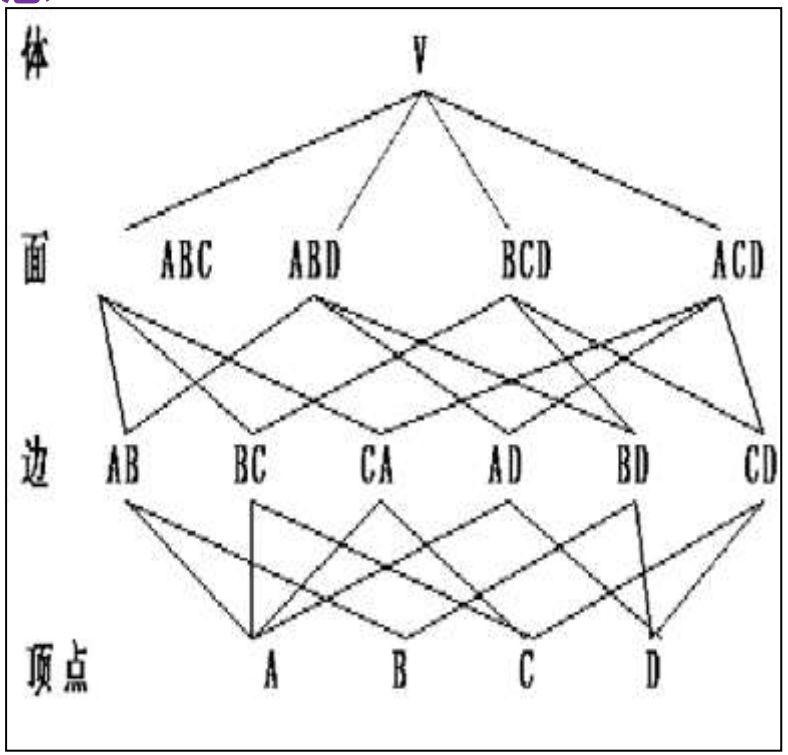
Properties of Geometric Objects(cont.)

- 图形对象的属性(cont.)
  - **非图形信息**:线型，颜色，亮度，质量，比重，体积等。
  - **图形信息**：对象和构成它的点、线、面的位置和相互关系，几何尺寸等。
    - 例如四面体：**几何信息**：顶点表、边表、多边形表 + **拓扑信息**



四面体ABCD

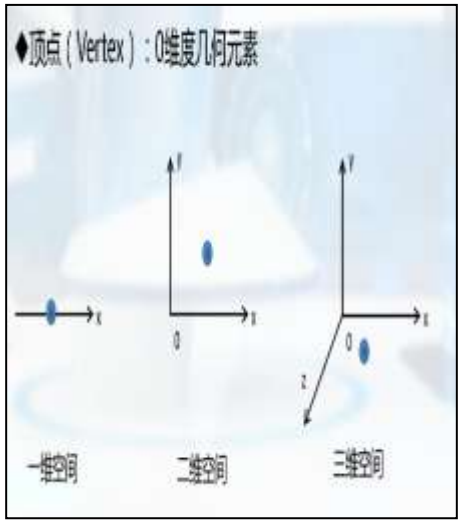




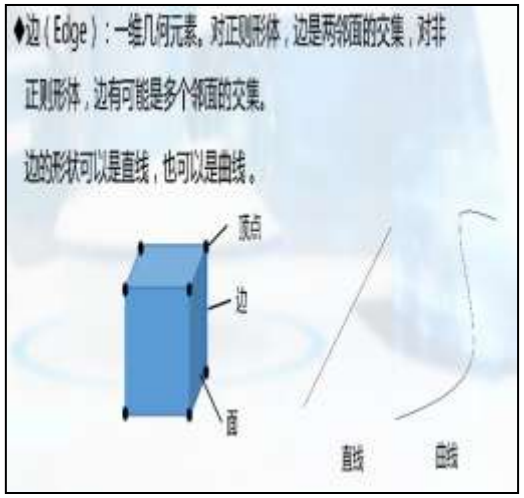
Properties of Geometric Objects(cont.)

- 图形对象的构成（cont.）
  - 基本几何元素的表示维度*

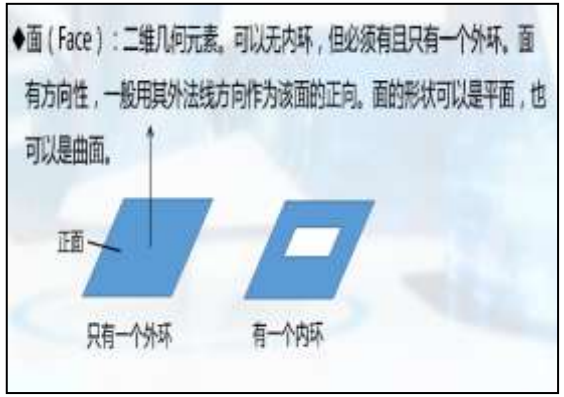
点P: 0维



线P(u)：1维

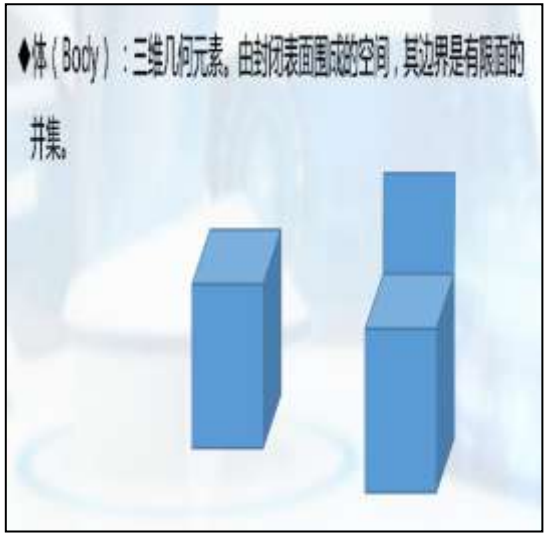


面/环P(u,v)：2维





体P(u,v,n)：3维



Outline

- Properties of Geometric Objects几何对象的属性
- Geometry Objects Representation几何对象的表示
- Modeling Technology几何造型技术
  - 实体造型技术Solid Modeling*
    - CSG(Constructive Solid Geometry ）构造实体几何
    - SP(Space-Partitioning Representation）空间分割表示
    - B-reps(Boundary Representation)边界表示*
      - Mesh网格*
      - Spline Curve and Surface 样条/*
  - 过程建模Process Modeling
    - Fractal geometry分形几何
    - Particle system粒子系统

Geometry Objects Representation（cont.）

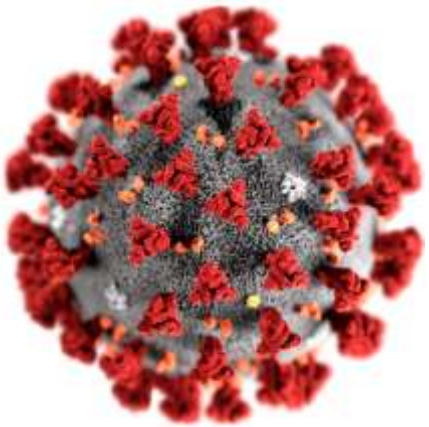
- Problem：研究如何在计算机中“表示”不同的图形对象？
  - 参： GAMES101-现代计算机图形学入门-闫令琪: lec10,lec11,lec12 Geometry





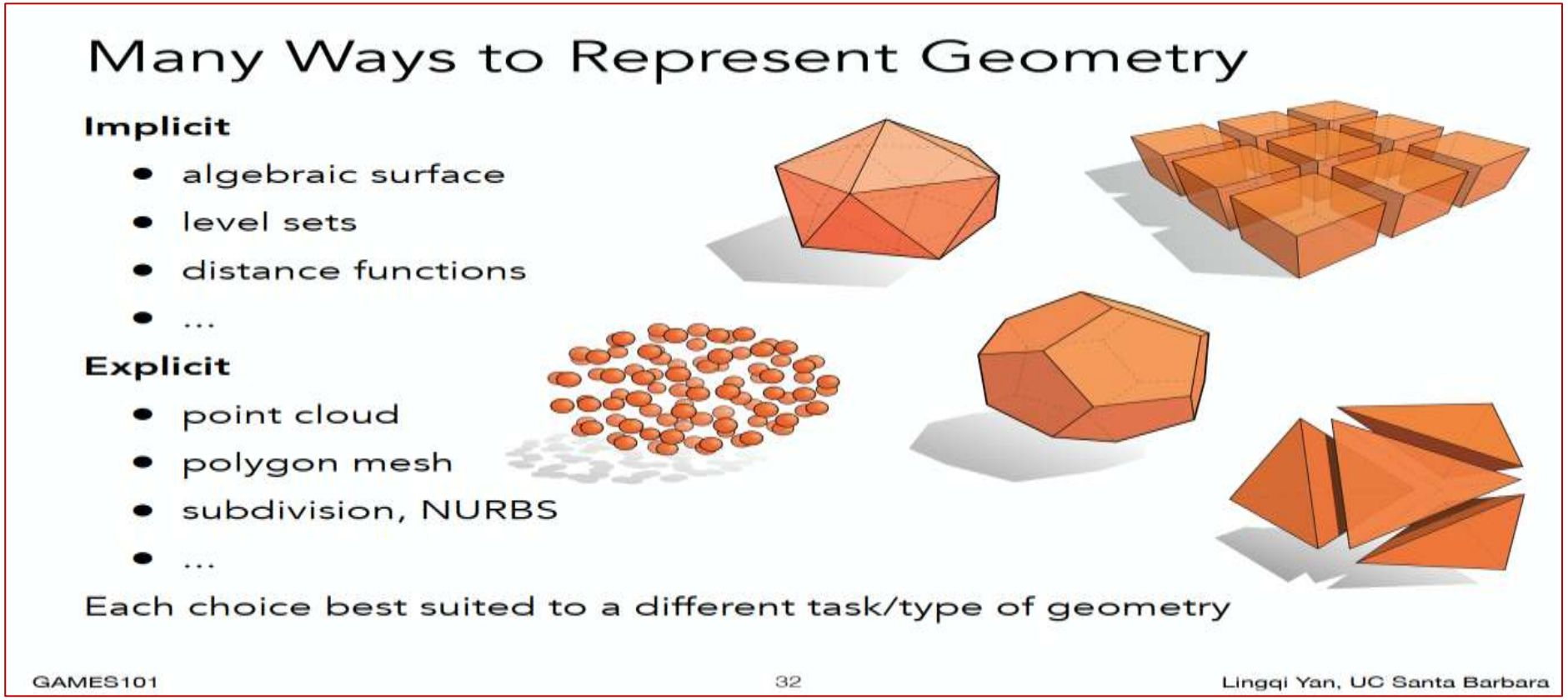






Geometry Objects Representation（cont.）

1. Implicit（隐式）: 代数表面，水平集，距离函数...
2. Explicit（显式）: 点云，多边形网格，细分曲面，NURBS...



Geometry Objects Representation（cont.）

- Implicit（隐式）：代数表面，水平集，距离函数...
- Explicit（显式）：点云，多边形网格，细分曲面，NURBS...

隐式表达优点

- 点在图形内外的判断简单（符号判断）
- 求交、布尔运算、融合方便
- 法向量计算简单（梯度）
- 适合描述复杂拓扑和体数据

隐式表达缺点

- 不易直接生成图形上的点
- 局部修改困难
- 参数化困难

圆的两种表达

- 隐式：$x^2 + y^2 - 1 = 0$
- 显示：$(\cos t, \sin t),\ t \in [0, 2\pi)$

球面的两种表达

- 隐式：$x^2 + y^2 + z^2 - 1 = 0$
- 显示：$(\sin\phi\cos\theta, \sin\phi\sin\theta, \cos\phi)$

直线

- 隐式：$ax + by + c = 0$
- 显示：$(x_0 + at,\ y_0 + bt)$

显示表达优点

- 容易采样和生成点
- 有自然参数域，适合纹理映射
- 局部控制灵活（尤其样条曲线曲面）
- 适合直接渲染和网格生成

显示表达缺点

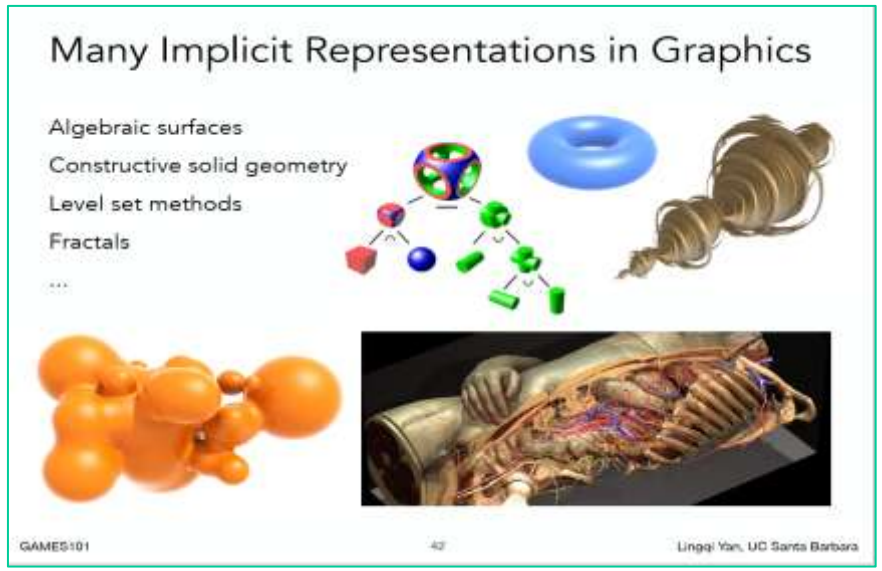
- 判断内外较麻烦
- 求交和布尔运算复杂
- 拓扑变化处理困难
- 有时难以精确表示某些隐式曲面

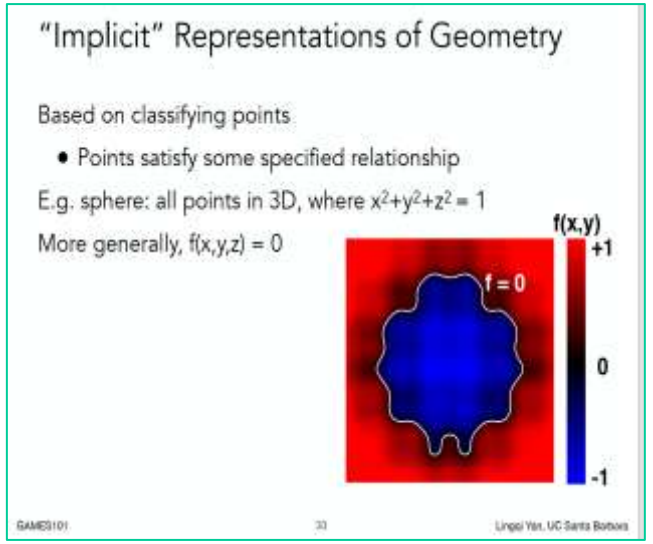
Geometry Objects Representation（cont.）

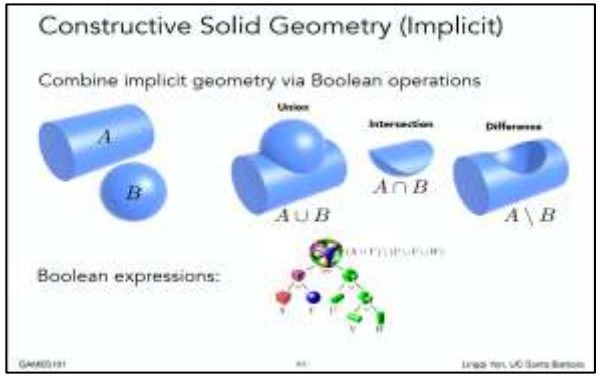
1. Implicit Representation（隐式表示）

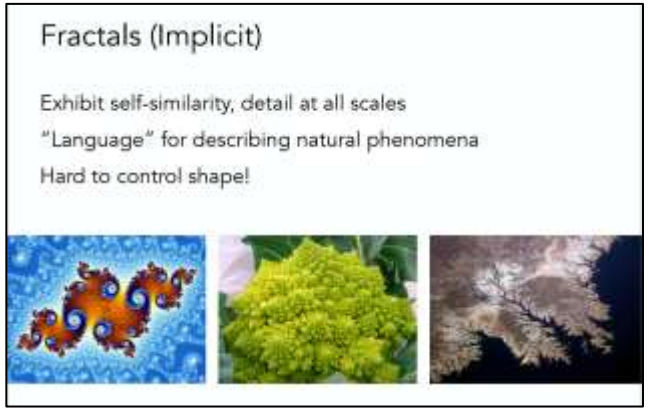
- 满足特定关系的点集：用一个方程或不等式来定义图形，通常形式为：$F(x, y, z) = 0$

  如：代数表面，构造实体几何CSG，水平集level set，符号距离场SDF，分形fractal









Geometry Objects Representation（cont.）

1. Implicit Representation（隐式表示）(续)

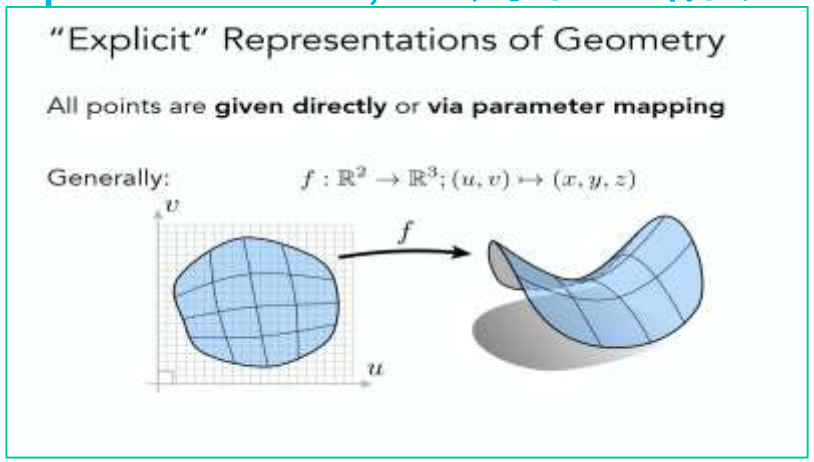
- 满足特定关系的点集：用一个方程或不等式来定义图形，通常形式为：$F(x, y, z) = 0$

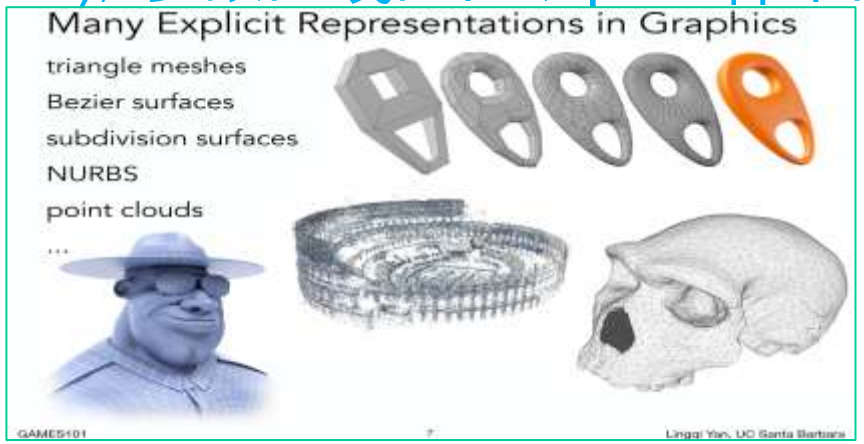
隐式类型	数学形式	典型场景
解析隐式	多项式/初等函数方程	光线求交、二次曲面、平面
符号距离场 SDF	$d(\mathbf{x}) = 0$	Shader 绘图、避障、字体
水平集	$\phi(\mathbf{x}, t) = 0$	流体、火焰、界面演化
标量场等值面	$f(\mathbf{x}) = c$	医学重建、科学可视化
布尔组合隐式	$\min$ / $\max$ 组合	CSG、SDF 布尔
程序化隐式	噪声/分形函数	地形、分形、程序化建模
神经隐式	$F_\theta(\mathbf{x}) = 0$	NeRF、DeepSDF、神经渲染

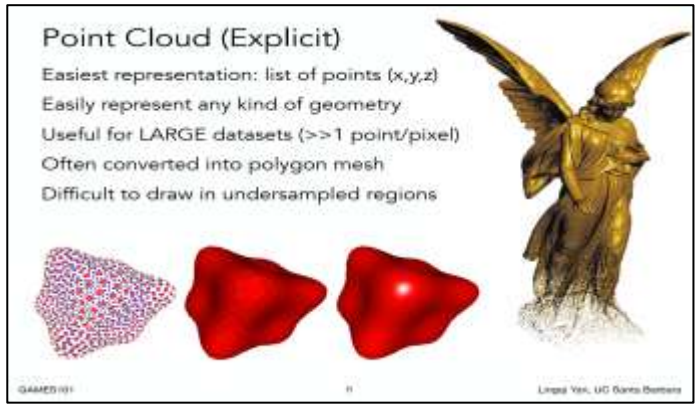
Geometry Objects Representation（cont.）

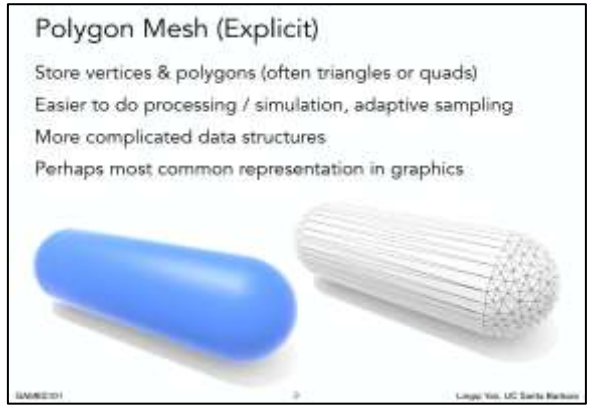
2. Explicit Representations (显式表示)

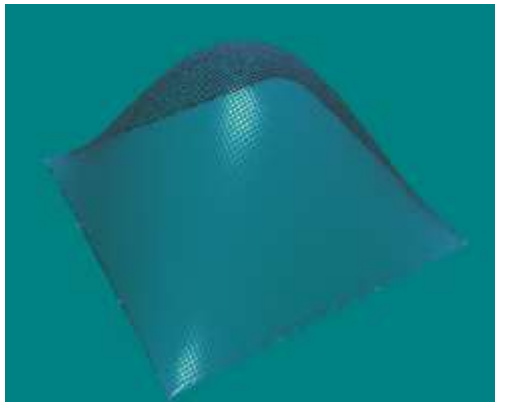
- 直接用参数或函数把坐标表示出来，通常形式为：$\mathbf{P}(u,v) = (x,(u,v),y(u,v)z(u,v))$
- 如：点云point Cloud，三角形网格（mesh），参数曲线曲面（spline样条曲线）











Geometry Objects Representation（cont.）

2. Explicit Representations (显式表示)

- 直接用参数或函数把坐标表示出来，通常形式为：$\mathbf{P}(u,v) = (x,(u,v),y(u,v)z(u,v))$
  - 如：点云point Cloud，三角形网格（mesh），参数曲线曲面（spline样条曲线）

显示类型	数学形式	典型场景
参数曲线	$\mathbf{P}(t) = (x(t), y(t), z(t))$	字体、路径动画、机器人轨迹
参数曲面	$\mathbf{P}(u,v) = (x(u,v), y(u,v), z(u,v))$	CAD、自由曲面、纹理映射
有理参数曲线曲面	NURBS 有理形式	工业设计、精确解析形状
参数化网格	顶点+参数/权重	角色动画、变形、蒙皮
参数化点/高斯	点+特征+参数	3D Gaussian Splatting
扫掠/旋转参数化	低维参数组合生成	管道、瓶罐、建筑构件
变形场参数化	$\mathbf{x}' = \mathbf{T}_\theta(\mathbf{x})$	医学配准、自由变形
时间参数化轨迹	$\mathbf{P}(t)$, t 为时间	动画、机器人、相机路径

Outline

- Properties of Geometric Objects 图形对象的属性
- Geometry Objects Representation 几何对象的表示
- **Modeling Technology造型技术**
  - 实体造型技术Solid Modeling*
    - CSG(Constructive Solid Geometry)构造实体几何
    - SP(Space-Partitioning Representation)空间分割表示
    - B-reps(Boundary Representation)边界表示*
      - Mesh网格*
      - Spline Curve and Surface 样条/*
  - 过程建模Process Modeling
    - Fractal geometry分形几何
    - Particle system粒子系统

Modeling Technology

- **Modeling Technique造型技术**
  - 研究”如何建立恰当的模型以表示不同图形对象”的技术，也称为CAGD（Computer Aided Geometric Design）计算机辅助几何设计
  - “Best Representation Depends on the Task” 根据任务建立最好的表示

- 造型技术分类（根据是否构造实体进行分类）
  - “几何形体”：如点、直线、曲线、平面、曲面，实体对象



“实体建模”（通过几何元素(点线面体)构造几何形体）

  - “自然形体”：如山、水、树、草、云、烟等自然界丰富多彩的对象



“过程建模”（通过算法和规则来自动生成几何结构）

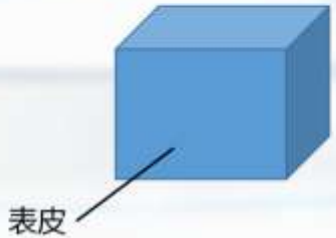
Outline

- Properties of Geometric Objects几何对象的属性
- Geometry Objects Representation几何对象的表示
- **Modeling Technology几何造型技术**
  - **实体造型技术Solid Modeling***
    - **CSG(Constructive Solid Geometry)构造实体几何**
    - **SP(Space-Partitioning Representation)空间分割表示**
    - **B-reps(Boundary Representation)边界表示***
      - **Mesh网格***
      - **Spline Curve and Surface 样条/***
  - **过程建模Process Modeling**
    - **Fractal geometry分形几何**
    - **Particle system粒子系统**

Solid Modeling(实体建模)

➢实体

三维空间中的物体是一个内部连通的三维点集。形象地说，是由其内部的点集及紧紧包着这些点的表皮组成的。



实体的性质：

(1)刚性：必须具有一定的形状；

(2)维数的一致性：三维空间中，一个物体的各部分均应是三维的；

(3)占据有限的空间：体积有限；

(4)边界的确定性：根据物体的边界能区别出物体的内部及外部；

(5)封闭性：经过一系列刚体运动及任意序列的集合运算之后，仍然是有效的物体。

物体表面的性质：

(1)连通性：位于物体表面上的任意两个点都可用实体表面上的一条路径连接起来；

(2)有界性：物体表面可将空间分为互不连通的两部分，其中一部分是有界的；

(3)非自相交性：物体的表面不能自相交；

(4)可定向性：表面的两侧可明确定义出属于物体的内侧或外侧；

(5)闭合性：物体表面的闭合性是由表面上多边形网格各元素的拓扑关系决定的。



克莱茵瓶（Klein Bottle）
自交且不可定向的封闭曲面



莫比乌斯带（Mobius Band）
单边不可定向

Solid Modeling(实体建模)

实体造型方法的主要三类方法：

1. 构造实体几何（Constructive Solid Geometry，CSG）

   它将实体表示成立方体、长方体、圆柱体、圆锥体等基本体素的组合，可以采用并、交、差等运算构造 新的形体。

2. 空间分割表示（Space-Partitioning Representation, SP）

   用来描述物体的内部性质，将包含物体 的空间区域划分成一组小的、非重叠的、连续实体（通常是立方体），由这些小实体集合构成。又叫做“分解表示法”

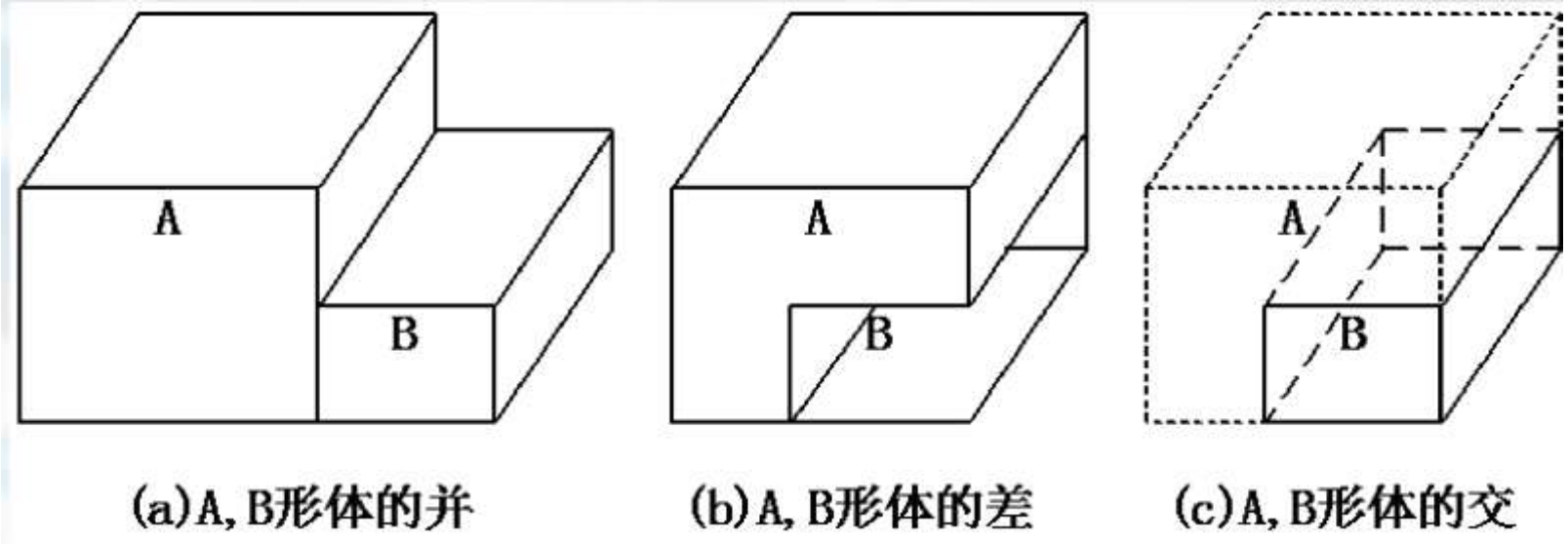
3. 边界表示（Boundary Representation, B-reps）*

   即用一组曲面（含平面）来描述物体，这些曲面将物体分为内部和外部。

   边界表示具体又包括多边形表面模型mesh和扫描表示两种。

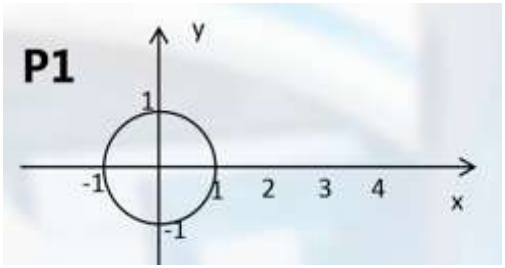
Solid Modeling（实体建模）-1.CSG

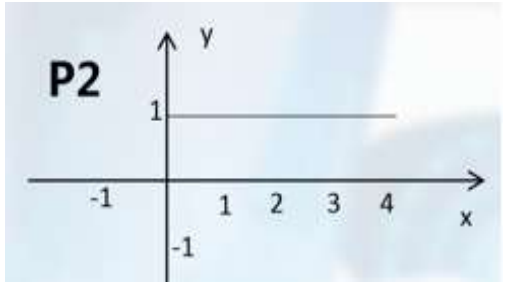
- **1)构造实体几何(CSG)**
  - 构造实体几何表示（Constructive Solid Geometry，CSG）
  - 它将实体表示成 立方体、长方体、圆柱体、圆锥体等基本体素的组合，
  - 可以采用并、交、差等运算，构造新的形体。

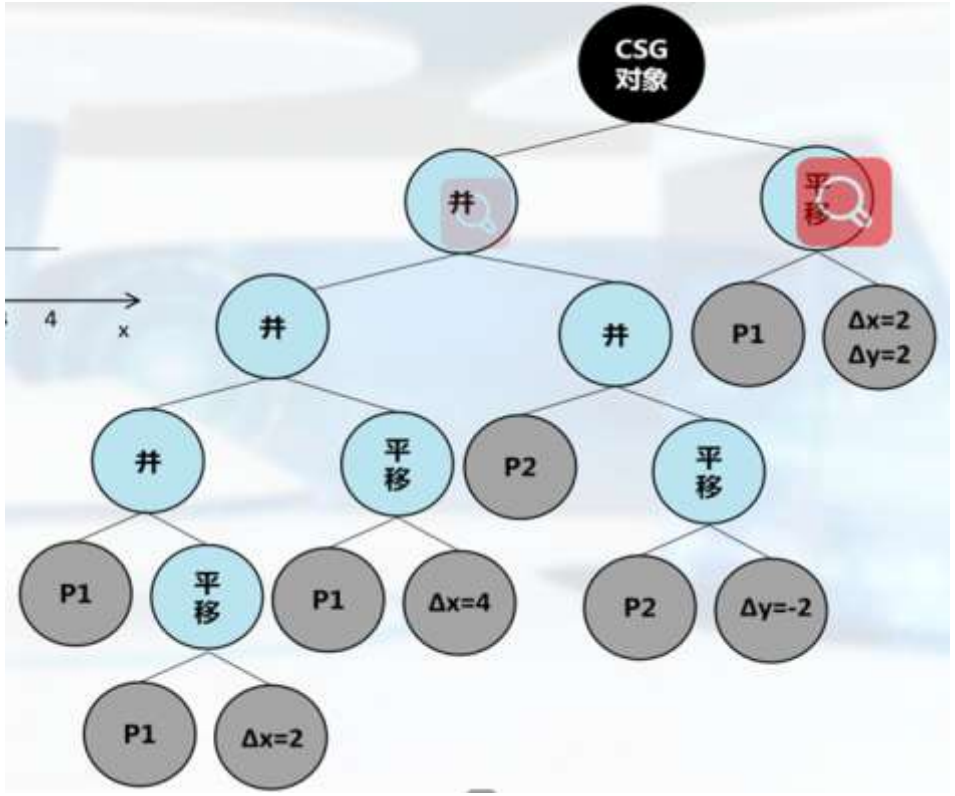


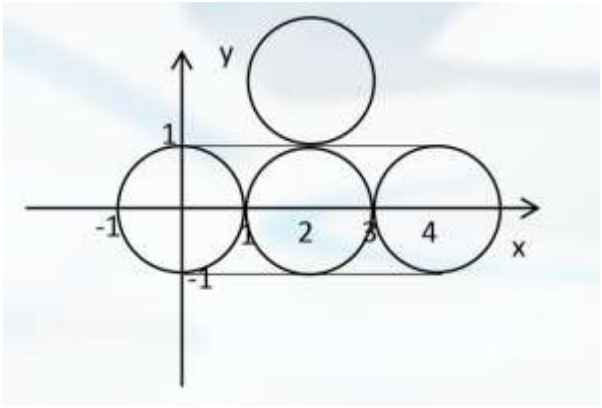
Solid Modeling（实体建模）-1.CSG(cont.)

- **1)构造实体几何表示(CSG)~二维实例**
  - 集合运算的实现过程可以用一棵二叉树（称为CSG树）
    - 叶子结点：基本体素或变换参数
    - 非叶子结点：施加于子结点的正则集合算子或几何变换定义
    - 根结点：最终结果：实体









Solid Modeling -1.CSG(cont.)

1)构造实体几何表示(CSG)-优缺点分析

优点：如果体素设置比较齐全，通过集合运算就可构造出多种不同的符合需要的实体。

缺点：当用户输入体素时，主要是给定体素的有关参数，然后由系统给出该体素的表面方程，再由系统进行集合的求交运算，最后得到生成的实体。这里存在两个问题：

1）集合运算的中间结果难以用简单的代数方程表示，求交困难；

2）CSG树不能显式地表示形体的边界，因而无法直接显示CSG树表示的形体。边界显示代价大，边界求法可采用“光线投射法”来直接找。

光线投射算法的核心思想：

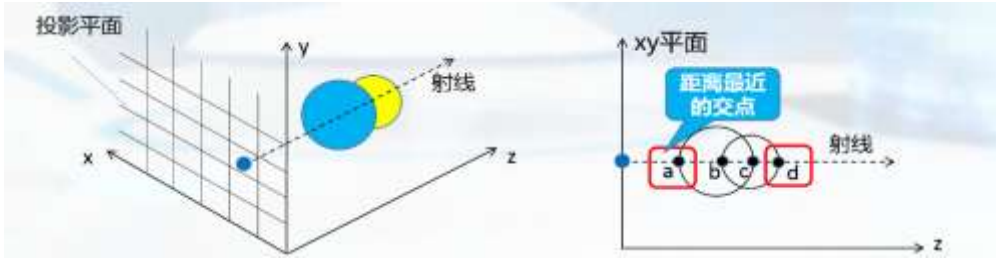
- **从显示屏幕（投影平面）的每一象素位置发射一根光线（射线），**
- **求出射线与距离投影平面最近的可见表面的交点和交点处的表面法矢量。**

(1)将射线与CSG树中的所有基本体素求交，求出所有的交点；

(2)将所有交点相对于CSG树表示的物体进行分类，确定位于物体边界上的那部分交点；

(3)对所有位于物体边界上的交点计算它们在射线上的参数值并进行排序，确定距离最近的交点。并得到其所在基本体素表面的法矢量。

- **然后根据光照模型 计算出表面可见点的色彩和亮度，生成实体的光栅图形。**

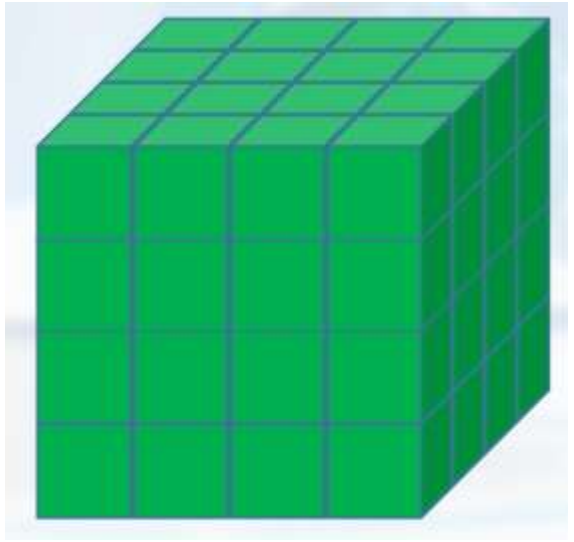


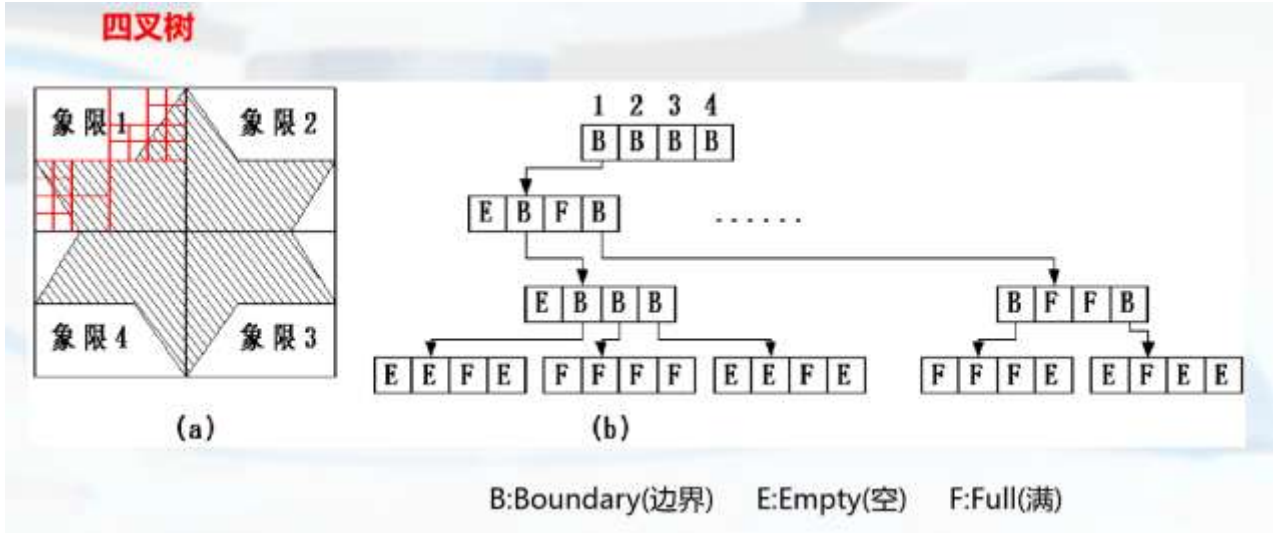
Outline

- Properties of Geometric Objects几何对象的属性
- Geometry Objects Representation几何对象的表示
- **Modeling Technology几何造型技术**
  - **实体造型技术Solid Modeling***
    - **CSG(Constructive Solid Geometry）构造实体几何**
    - **SP(Space-Partitioning Representation）空间分割表示**
    - **B-reps(Boundary Representation)边界表示***
      - **Mesh网格***
      - **Spline Curve and Surface 样条/***
  - **过程建模Process Modeling**
    - **Fractal geometry分形几何**
    - **Particle system粒子系统**

Solid Modeling -2.SP

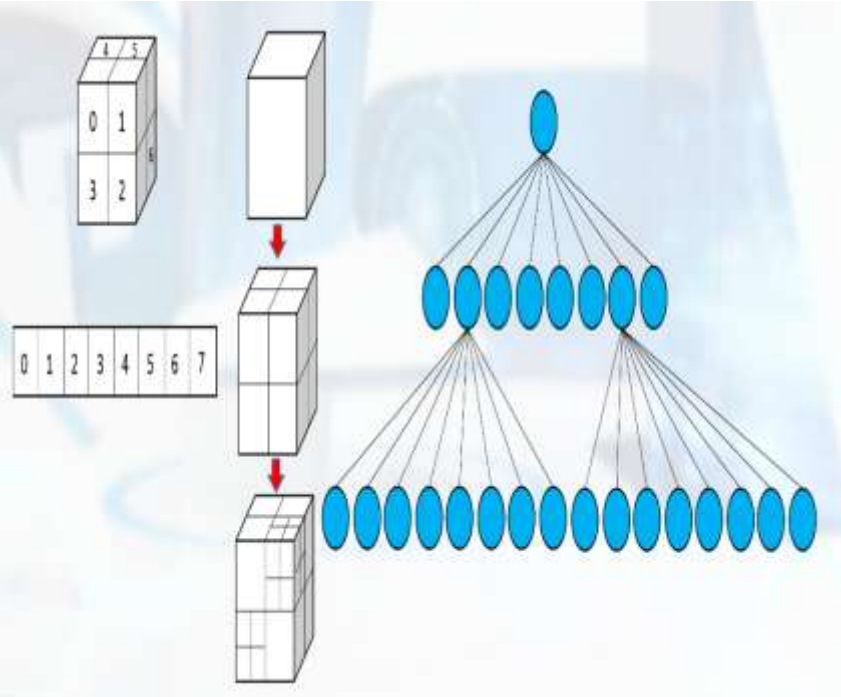
- 2.空间分割表示（Space-Partitioning Representation）
  - 用来描述物体的内部性质，将包含一物体 的空间区域划分成一组小的、非重叠的、连续实体（通常是立方体）：空间位置枚举表示.
    - 将包含实体的空间分割为大小相同、形状（如 正方形或立方体）的体素，然后，以体素的集合来表示图形对象。
    - 用三维数组P[I][J][K]表示物体，数组中的元素与单位小立方体一一对应。

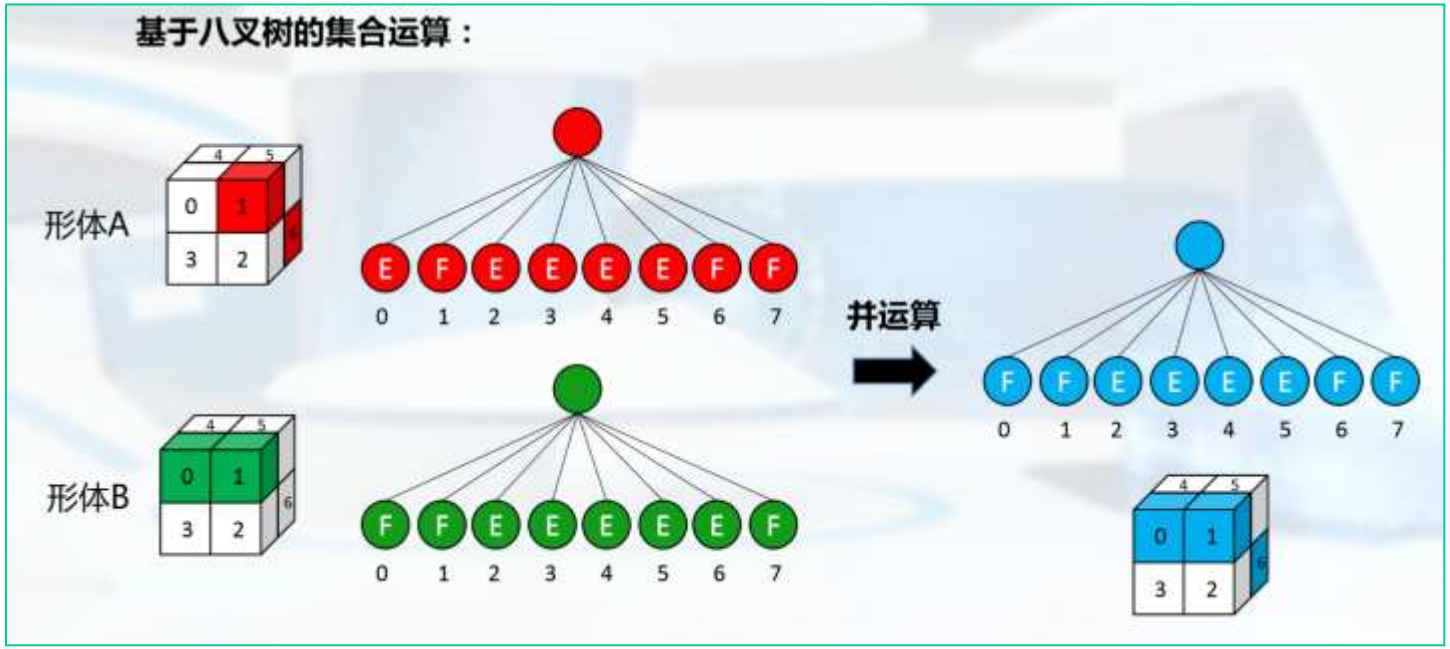




Solid Modeling -2.SP(cont.)

- 2.空间分割表示SP---**八叉树法（octrees）**
- 又称为分层树结构，它对空间进行**自适应划分**，采用具有层次结构的八叉树来表示实体。





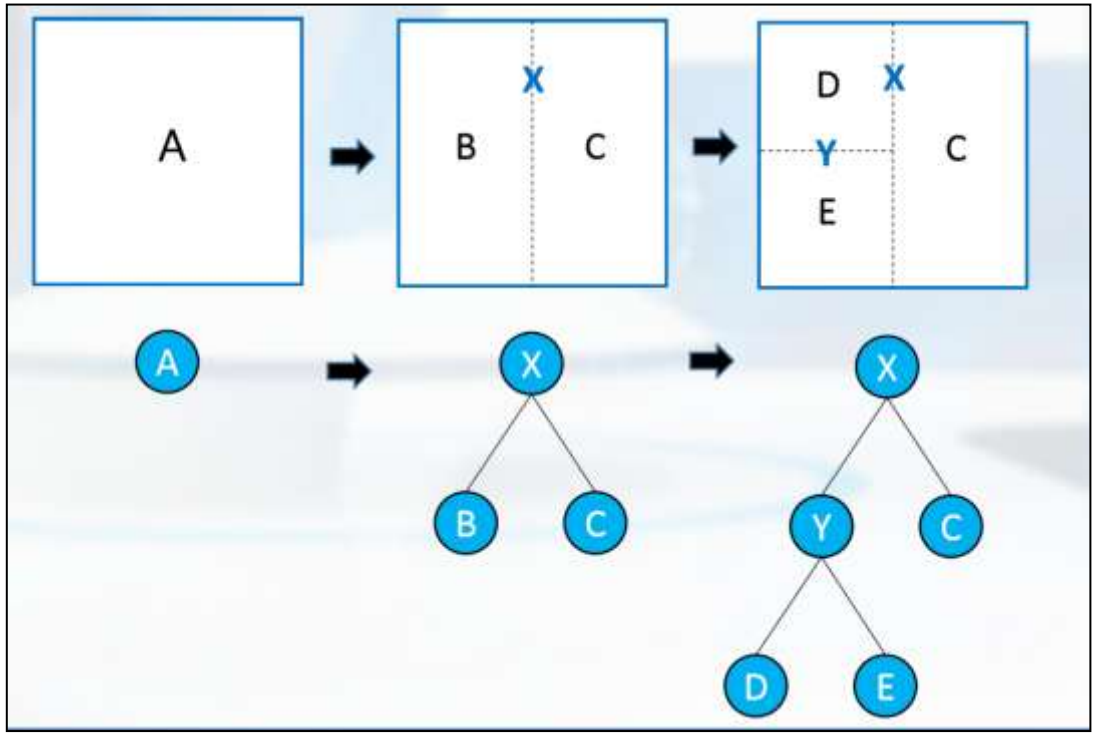
- *note: B: Boundary, E:Empty , F: Full*

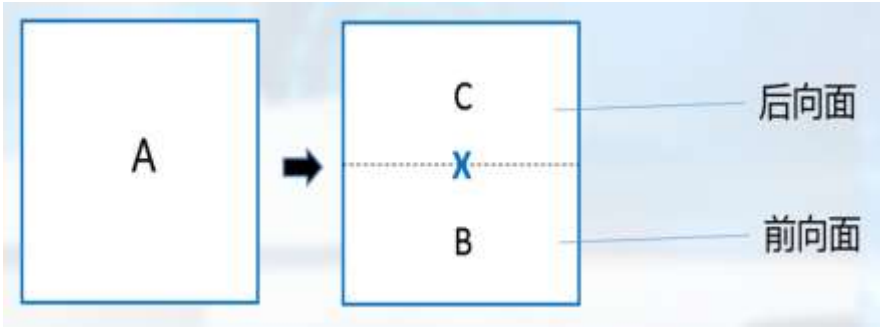
Solid Modeling -2.SP(cont.)

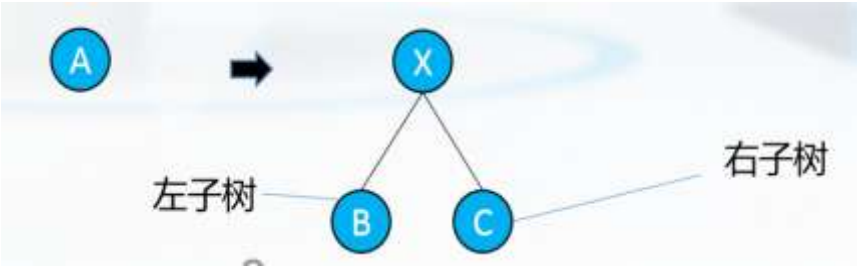
- 2)-空间分割表示(SP)---二叉空间分割法(BSP)

  二叉空间分割（Binary Space Partitioning，BSP）每次将一实体用任一位置和任一方向的平面分为二部分来表示，是高效的分割表示法。

  自适应分割：BSP树可以减少场景树的深度,减少搜索时间；

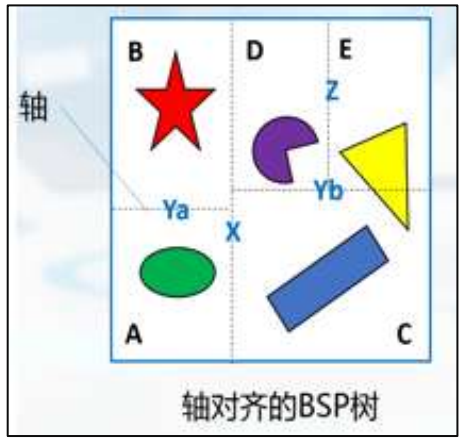




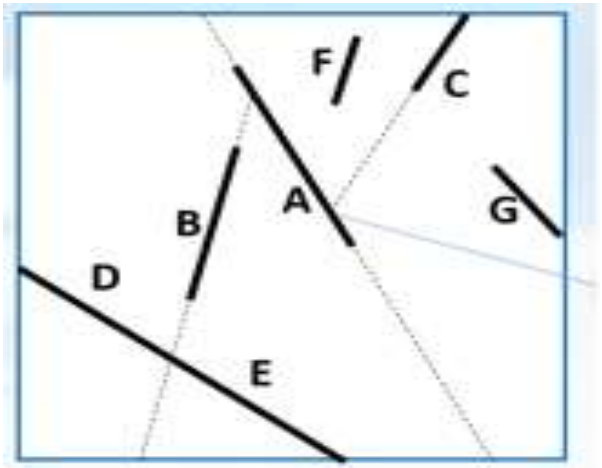


Solid Modeling -2.SP(cont.)

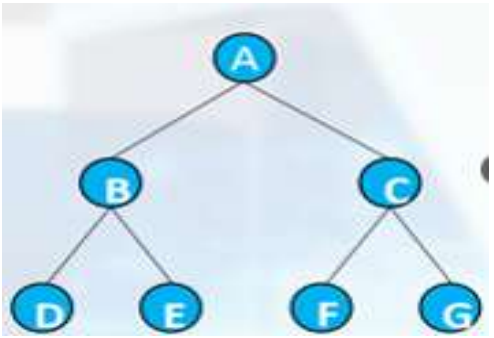
- 2)-空间分割表示(cont.)---**二叉空间分割法(BSP)（cont.)**
  - BSP树的两种类型：
    - 轴对齐BSP树：只能对应粗排序.不常用。
    - 多边形对齐BSP树：对应相对视点的排序，可与后面将要介绍的画家算法(一种消隐算法) 配合，进行场景中物体的**深度测试**，还可以作**相交测试**，**碰撞检测**



轴对齐的BSP树



多边形对齐的BSP树

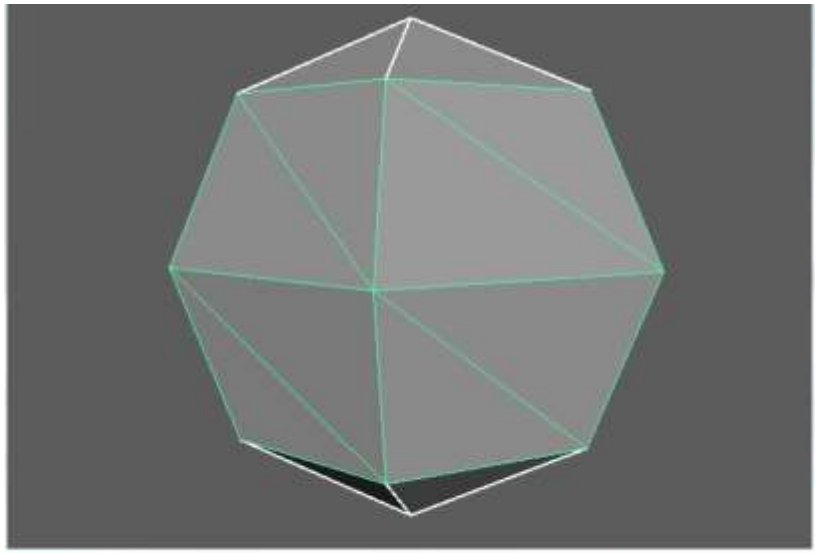


Outline

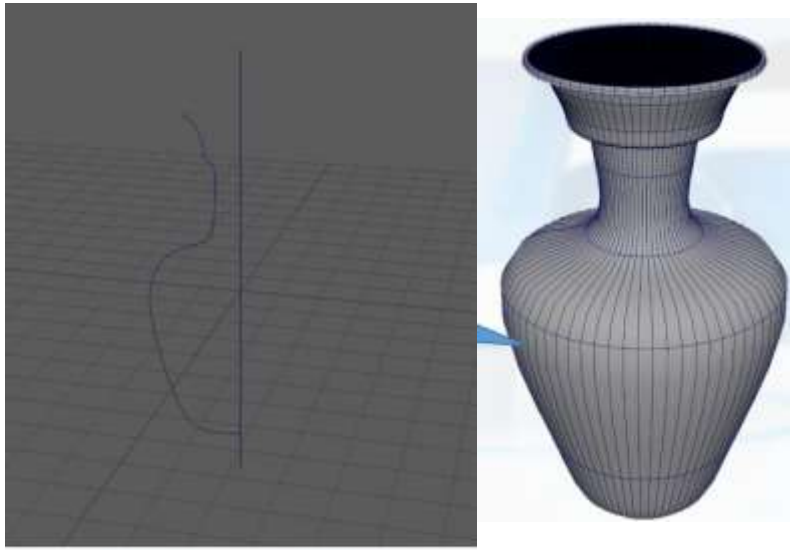
- Properties of Geometric Objects几何对象的属性
- Geometry Objects Representation几何对象的表示
- **Modeling Technology造型技术**
  - **实体造型技术Solid Modeling***
    - CSG(Constructive Solid Geometry)构造实体几何
    - SP(Space-Partitioning Representation)空间分割表示
    - **B-reps(Boundary Representation)边界表示***
      - Mesh网格*
      - Spline Curve and Surface 样条/*
  - **过程建模Process Modeling**
    - Fractal geometry分形几何
    - Particle system粒子系统

Solid Modeling -3.B-reps

- 3.边界表示(Boundary Representation, B-reps)*
  - 用一组曲面(含平面)来描述物体!(被大量采用的造型方法)
    - 扫描表示模型
    - 多边形表面模型*(重点介绍和掌握)



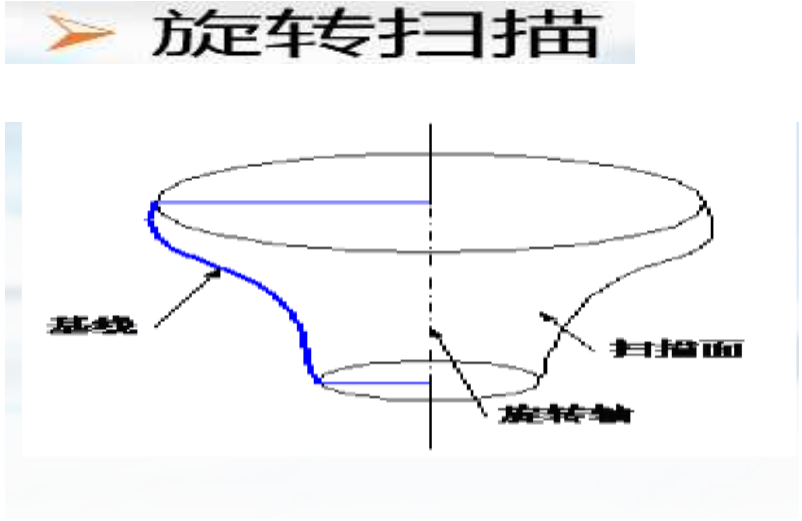
多边形表面模型



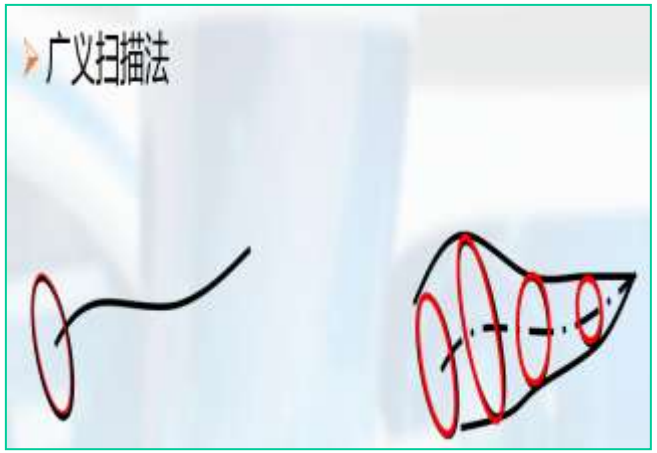
扫描表示

Solid Modeling -3.B-reps(cont.)

- **3. 边界表示(B-reps)---扫描表示法**
  - 将空间中的一个点、一条边或一个面沿某一路径扫描时，所形成的轨迹将定义一个“实体”。包含两个要素：
    - 一是作扫描运动的基本图形（基线）:后面介绍样条曲线曲面生成
    - 二是扫描运动的方式，如旋转扫描（圆形-轴对称），非圆形扫描（平移，螺旋），广义扫描

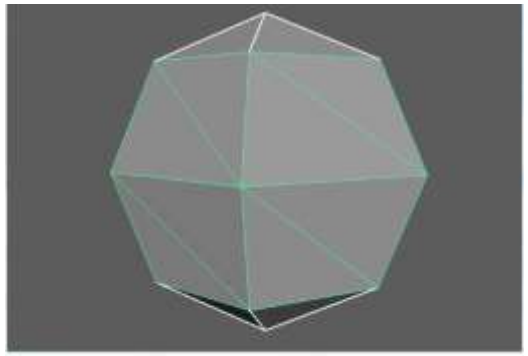




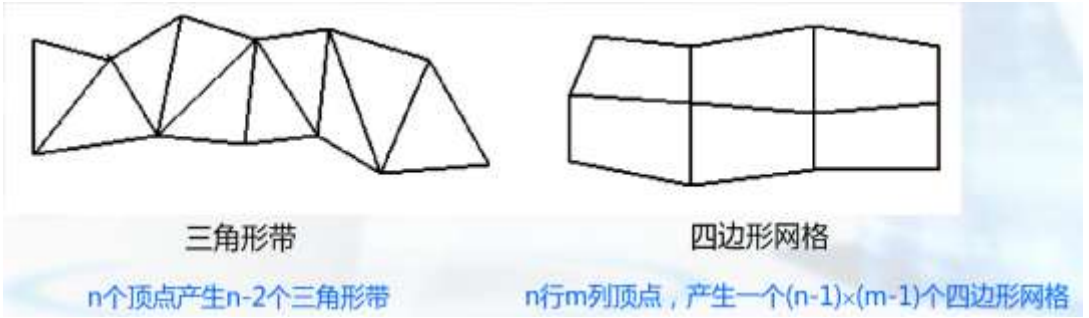


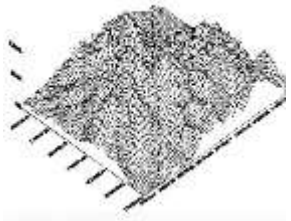
Solid Modeling -3.B-reps(cont.)

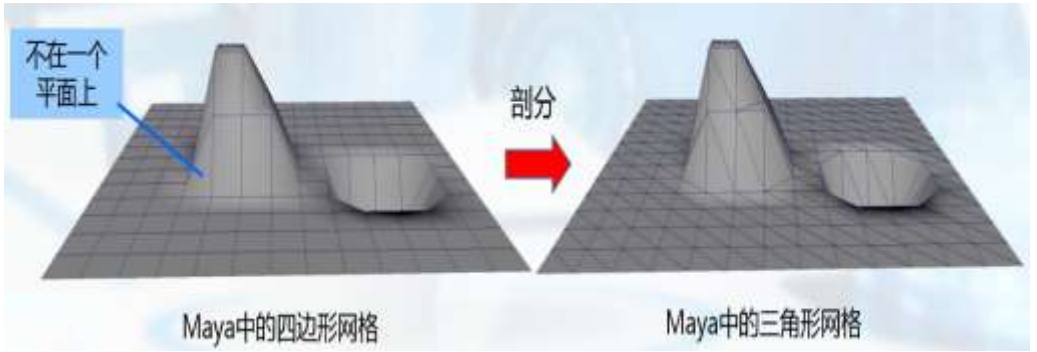
- **3. 边界表示(B-reps)---多边形表面表示**
  - Polygon Surface Model使用一组包围物体内部的平面多边形来描述**实体**。
    - “三角形网格triangle mesh”





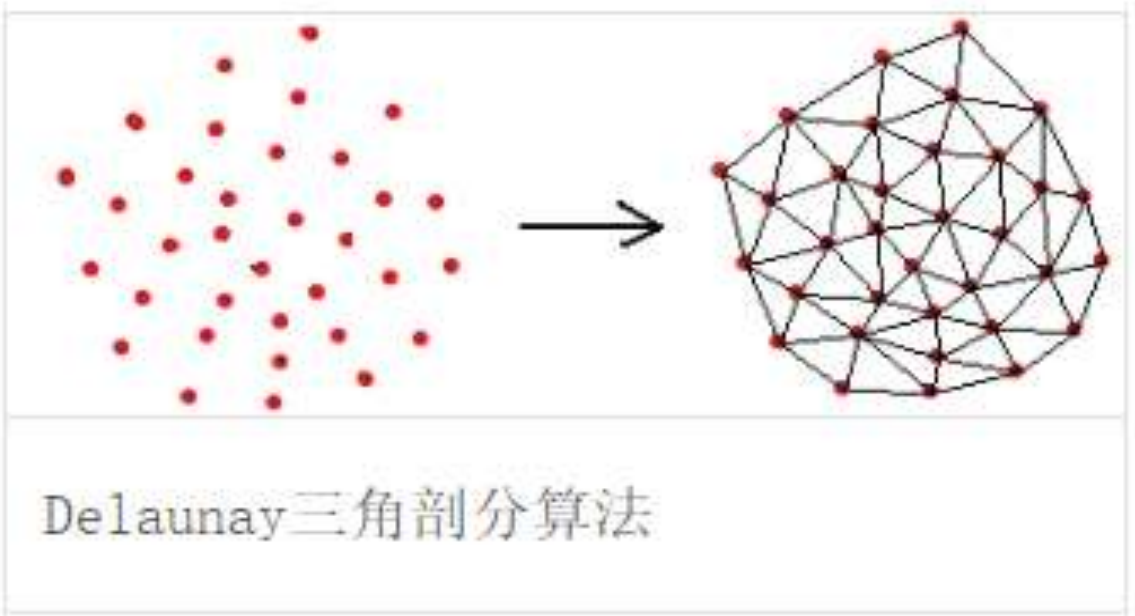






Solid Modeling -3.B-reps(cont.)

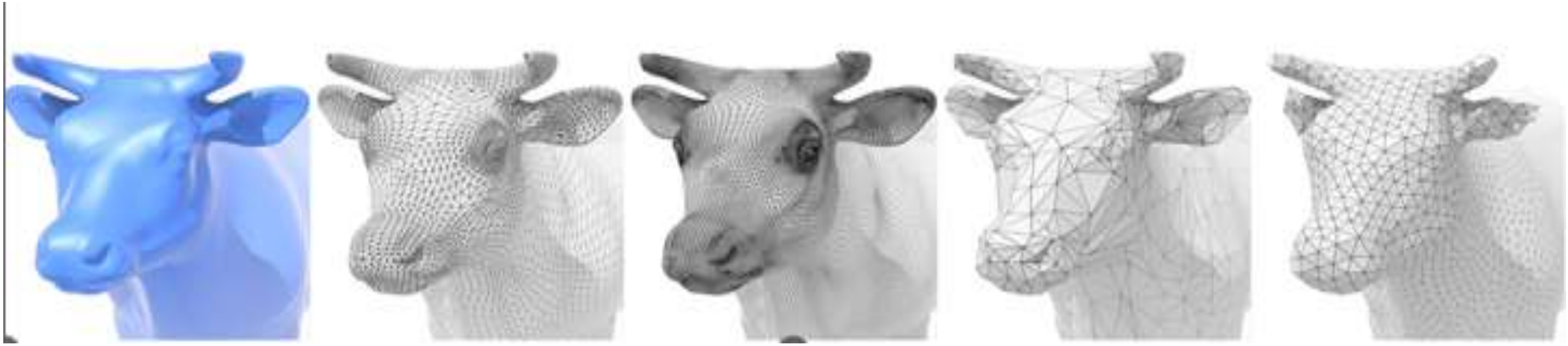
- **3. 边界表示(B-reps)---多边形表面模型(续)**
  - ➢ 点集的三角剖分(Triangulation)
    - 参教材Chapter 12: Delaunay triangulation(狄洛尼三角剖分/德劳内三角化)算法
      - **关于点集的很多种几何图都和Delaunay三角剖分相关，如Voronoi图，EMST树，Gabriel图等**
      - **Delaunay三角剖分有最大化最小角，“最接近于规则化的“的三角网和唯一性(任意四点不能共圆)两个特点**



Solid Modeling -3.B-reps（cont.）

- **3. 边界表示(B-reps)---多边形表面模型（续）**
  - 网格优化技术：“网格细分Mesh **Subdivision**”：
    - ref GAME101 lecture12：*Mesh Operations: Geometry Processing*
      - **mesh subdivision:**通过增加三角形的数量，使要表示的曲面更加光滑。
      - **Mesh simplification:**用少一点的网格数量，去掉一些边和三角形，但仍维持相互关系
      - **Mesh regularization:三角形正则化：让某些**三角形（太大太小或细长）变得更像正三角形

(0) final picture (2) mesh subdivision (3) mesh simplification (4) mesh regularization



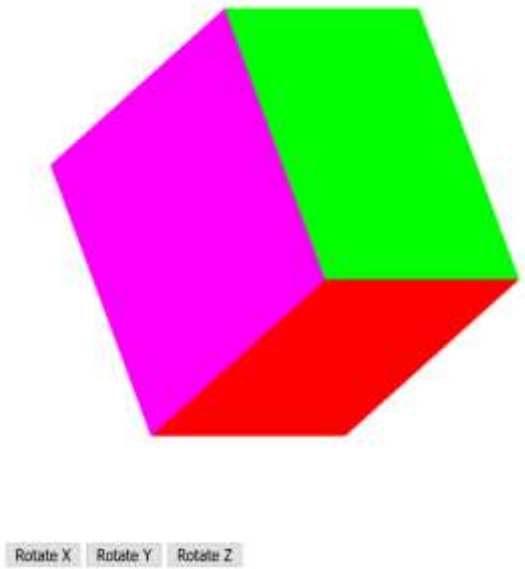
Outline

- Properties of Geometric Objects几何对象的属性
- Geometry Objects Representation几何对象的表示
- **Modeling Technology造型技术**
  - **实体造型技术Solid Modeling***
    - CSG(Constructive Solid Geometry)构造实体几何
    - SP(Space-Partitioning Representation)空间分割表示
    - **B-reps(Boundary Representation)边界表示***
      - **Mesh网格***
      - Spline Curve and Surface 样条曲线和曲面*
  - **过程建模Process Modeling**
    - Fractal geometry分形几何
    - Particle system粒子系统

Solid Modeling /3.B-reps /mesh

网格mesh：使用一组包围物体内部的平面多边形（常用三角形）来描述**实体.**

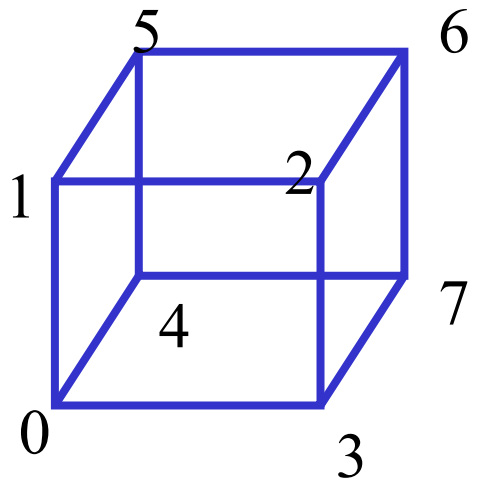
实例1：立方体网格建模 参见\Angle8E Code\04\cube.js, cubev.js

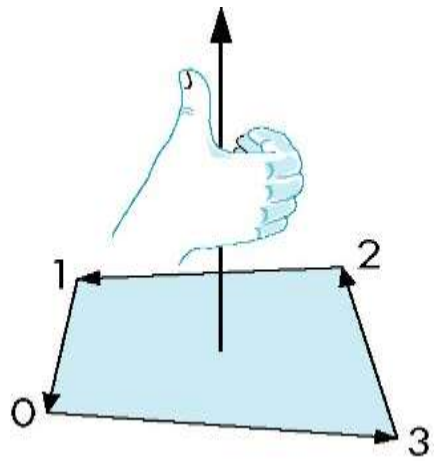


Solid Modeling /3.B-reps /mesh网格(cont.)

➢实例1:立方体网格建模 参见\Angle8E Code\04\cube.js

```
colorCube( )
{
    quad( 1, 0, 3, 2 ); //Z正-前
    quad( 4, 5, 6, 7 ); //Z负-后
    quad( 2, 3, 7, 6 ); //X正-右
    quad( 5, 4, 0, 1 ); //X负-左
    quad( 6, 5, 1, 2 ); //Y正-上
    quad( 3, 0, 4, 7 ); //Y负-下
}
```





- **Inward and Outward Facing Polygons(内向面和外向面多边形)**
  - The order $\{v_4, v_5, v_6, v_7\}$ and $\{v_6, v_7, v_4, v_5\}$ are equivalent in that the same polygon will be rendered, describe *outwardly facing* polygons(外向面多边形)
  - the order $\{v_5, v_4, v_7, v_6\}$ and $\{v_7, v_6, v_5, v_4\}$ are equivalent , describe *inward facing* polygons(内向面多边形)

➢ **"right-hand rule": counter-clockwise encirclement of outward-pointing normal**
(右手法则:"逆时针方向"指向该多边形的外向面的"外法向量方向")

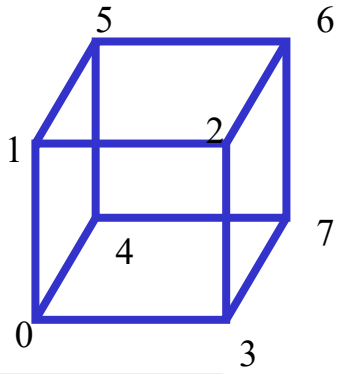
◆ OpenGL can treat inward and outward facing polygons differently

Solid Modeling /3.B-reps /mesh网格(cont.)

实例1:立方体网格建模 参见\Angle8E Code\04\cube.js (cont.)

```
var vertices = [
        vec3( -0.5, -0.5,  0.5 ),
        vec3( -0.5,  0.5,  0.5 ),
        vec3(  0.5,  0.5,  0.5 ),
        vec3(  0.5, -0.5,  0.5 ),
        vec3( -0.5, -0.5, -0.5 ),
        vec3( -0.5,  0.5, -0.5 ),
        vec3(  0.5,  0.5, -0.5 ),
        vec3(  0.5, -0.5, -0.5 )
    ];
```

```
function colorCube( )
{  quad( 1, 0, 3, 2 ); //Z正-前
    quad( 4, 5, 6, 7 ); //Z负-后
    quad( 2, 3, 7, 6 ); //X正-右
    quad( 5, 4, 0, 1 ); //X负-左
    quad( 6, 5, 1, 2 ); //Y正-上
    quad( 3, 0, 4, 7 ); //Y负-下}
```




```
var vertexColors = [
        [ 0.0, 0.0, 0.0, 1.0 ],  // black
        [ 1.0, 0.0, 0.0, 1.0 ],  // red
        [ 1.0, 1.0, 0.0, 1.0 ],  // yellow
        [ 0.0, 1.0, 0.0, 1.0 ],  // green
        [ 0.0, 0.0, 1.0, 1.0 ],  // blue
        [ 1.0, 0.0, 1.0, 1.0 ],  // magenta
        [ 0.0, 1.0, 1.0, 1.0 ],  // cyan
        [ 1.0, 1.0, 1.0, 1.0 ]   // white
    ];
```

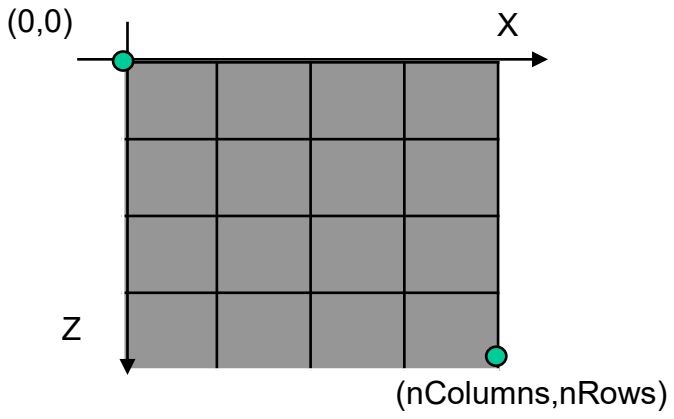
```
var quad(a, b, c, d)
{
   var indices = [ a, b, c, a, c, d ];
   for ( var i = 0; i < indices.length; ++i ) {
      points.push( vertices[indices[i]]);
      colors.push( vertexColors[indices[i]] );
    }
}
```

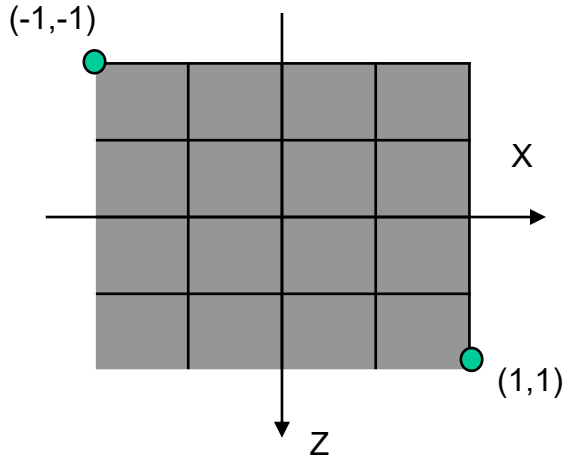
```
gl.drawArrays( gl.TRIANGLES, 0, points.length);
```

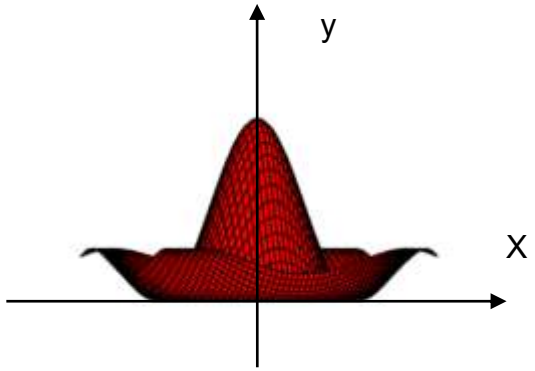
- 注意： 顶点顺序按外法向量方向
- 注意：每个四边形生成两个三角形 将两个三角形的位置和颜色数据放入数组中

Solid Modeling /3.B-reps /mesh网格(cont.)

- **实例2：四边形网格建模墨西哥帽子hat, 参见\Angle8E Code\05\ hat.***
- Step1: 先将(0,0),(nRow,nColomn)等距分割为(nColumns,nrows)个方格,再将坐标规约到(-1,-1)(1,1)
  - Zi=i * 2/(nColumns-1)-1　//例如下图中，nColumns=5，一般取奇数以保证图形对称
  - Xj=j * 2/(nRow-1)-1　　//例如下图中，nRow=5，　一般取奇数以保证图形对称
- Step2: 然后根据(x,z)，计算y值(高程)：y=f(z,x),存放到数组data中
  - y=(sin πr)/(πr),（r!=0, 非原点处, r=sqrt(x*x, z*z)
  - y=1,　　(r=0,中心点)
- Step3: 用前两部计算出来每个顶点的(x,y,z)，将所有顶点按顺序放顶点缓存再发送GPU绘制三角形







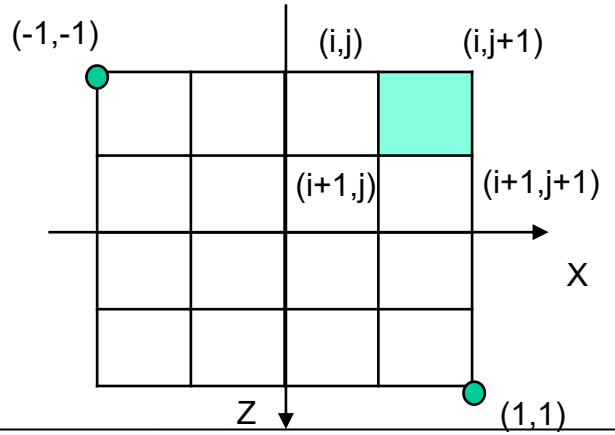
Solid Modeling /3.B-reps /mesh网格(cont.)

- **实例2**：墨西哥帽，参见\Angle8E Code\05\ hat.js (cont.)

1：先根据(z,x)的NDC坐标计算出y放data中

2：将每个方格四个顶点位置按“逆时针”顺序，放顶点位置数组中

3：用三角扇（每四个顶点绘制两三角形）进行渲染




```
// 根据行数列数，确定每个顶点的X,Y,Z值。
// 行Z:  Z=(2*i/nRows-1.0);
// 列X:  X=(2*j/nRows-1.0);
// 高度Y：计算出r=2*PI*sqrt(Z*Z+X*X),确定Y=sin(r)/r.
for(var i=0; i<nRows; i++) {
    var Z = (2*i/nRows-1.0);
    for(var j=0; j<nColumns; j++) {
        var X = (2*j/nRows-1.0);

        //r范围（-1，1）时，Y=sin(r)/r，发现只有帽子中心部分显示出来
        //var r = Math.PI*Math.sqrt(x*x+y*y);

        //r范围（-2pi,2pi)之间，Y=sin(r)/r，帽子全显。
        var r = 2*Math.PI*Math.sqrt(Z*Z+X*X);

        if(r) data[i][j] = Math.sin(r)/r;  else data[i][j] = 1;
    }
}
```

```
// vertex array of nRows*nColumns quadrilaterals（四边形）
// (two triangles/quad) from data
for(var i=0; i<nRows-1; i++) {
    for(var j=0; j<nColumns-1;j++) {
        pointsArray.push( vec4(2*i/nRows-1, data[i][j], 2*j/nColumns-1, 1.0));
        pointsArray.push( vec4(2*(i+1)/nRows-1, data[i+1][j], 2*j/nColumns-1, 1.0));
        pointsArray.push( vec4(2*(i+1)/nRows-1, data[i+1][j+1], 2*(j+1)/nColumns-1, 1.0));
        pointsArray.push( vec4(2*i/nRows-1, data[i][j+1], 2*(j+1)/nColumns-1, 1.0) );
}
```

```
// draw each quad as two filled red triangles
// and then as two black line loops
for(var i=0; i<pointsArray.length; i+=4) {
    gl.uniform4fv(fColor, flatten(red));
    gl.drawArrays( gl.TRIANGLE_FAN, i, 4 ); //按三角扇图元显示四边形红色填充面
    gl.uniform4fv(fColor, flatten(black));
    gl.drawArrays( gl.LINE_LOOP, i, 4 );//按线带图元显示四边形黑色边框
}
```

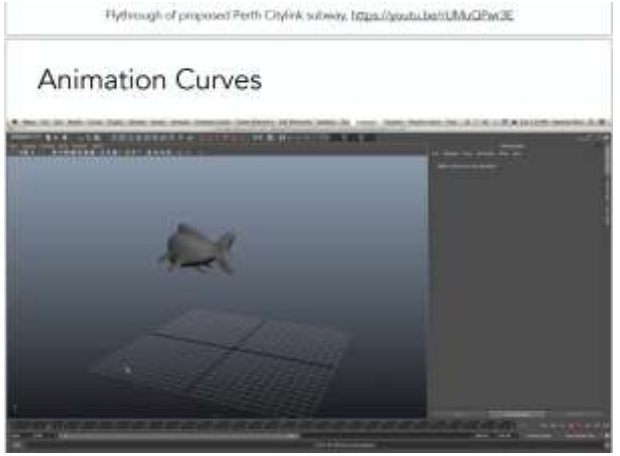
Outline

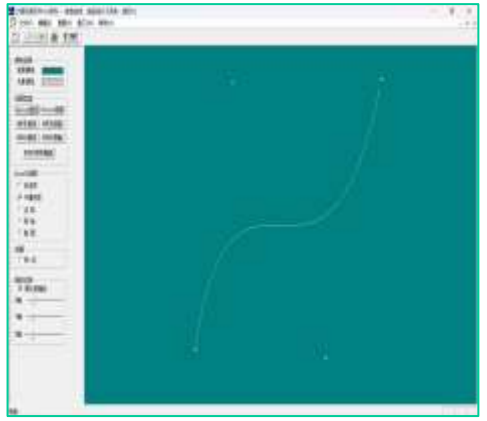
- Properties of Geometric Objects几何对象的组成
- Geometry Objects Representation几何对象的表示
- **Modeling Technology造型技术**
  - **实体造型技术Solid Modeling***
    - CSG(Constructive Solid Geometry)构造实体几何
    - SP(Space-Partitioning Representation)空间分割表示
    - **B-reps(Boundary Representation)边界表示***
      - Polygon Mesh 多边形网格*
      - **Spline Curve and Surface 样条曲线曲面***
  - **过程建模Process Modeling**
    - **Fractal geometry分形几何**
    - **Particle system粒子系统**

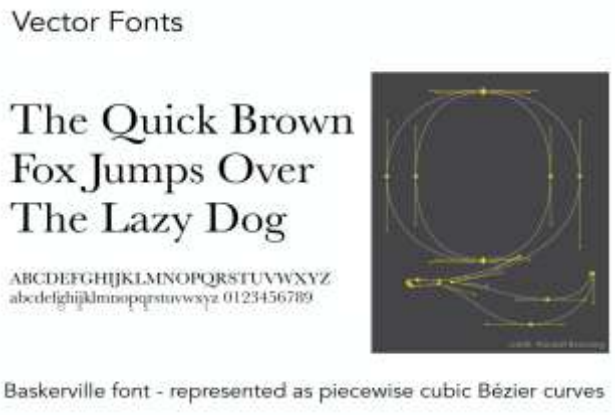
Solid Modeling /3.B-reps /Spline样条

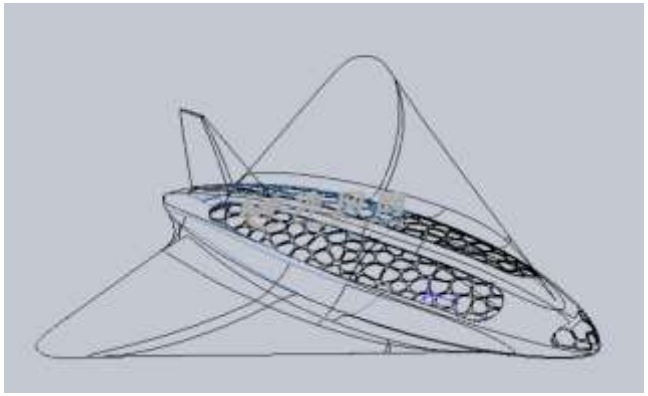
- Spline 样条曲线和曲面
  - 参数拟合曲线: “设计任意复杂形态的曲线曲面”
  - 应用场景 : 相机路径, 动画曲线, 向量字体, 造型边界表示, CAD……

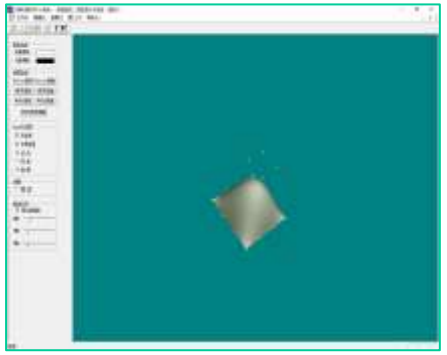












Solid Modeling /3.B-reps /Spline样条(cont.)

- **Spline** : curves designed under the control of a given set of points.

“样条”一词来源于 造船工程师 设计船体形状的“可弯曲的木条或者金属条”

- Spline
  - a continuous curve constructed so as to pass through a given set of points and have a certain number of continuous derivatives.
  - In short, a curve under control



A Real Draftsman’s Spline
http://www.alatown.com/spline-history-architecture/

Solid Modeling /3.B-reps /Spline样条（cont.）

The University of New Mexico

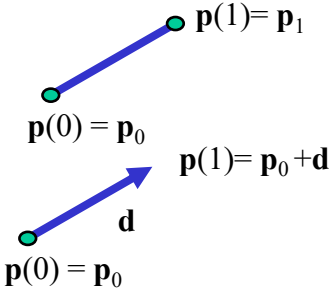
There are many ways to represent curves and surfaces, Want a representation is Stable 稳定, Smooth 平滑, Easy to evaluate 易于求值?

➢ 采用“参数化多项式表示”，可满足构造的图形稳定，平滑，易于求值等要求！

a. Parametric Lines: p(u)

- ➢Line connecting two points $\mathbf{p}_0$ and $\mathbf{p}_1$
- ➢normalize u to be over the interval (0,1)

$\mathbf{p}(u)=(1-u)\mathbf{p}_0+u\mathbf{p}_1$

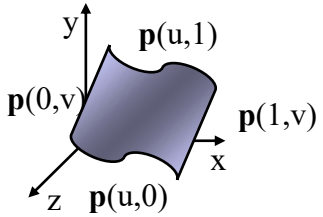


Ray from $\mathbf{p}_0$ in the direction $\mathbf{d}$

$\mathbf{p}(u)=\mathbf{p}_0+u\mathbf{d}$

b. Parametric Surfaces p(u,v)

- ➢p(u,v) = [x(u,v), y(u,v), z(u,v)]



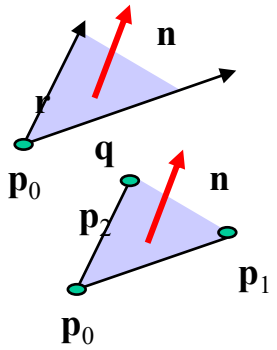
1. Parametric Planes参数平面

point-vector form

$\mathbf{p}(u,v)=\mathbf{p}_0+u\mathbf{q}+v\mathbf{r}$

three-point form

$\mathbf{q} = \mathbf{p}_1 - \mathbf{p}_0$
$\mathbf{r} = \mathbf{p}_2 - \mathbf{p}_0$
$\mathbf{n} = \mathbf{q} \times \mathbf{r}$
$\mathbf{P(u,v)=p0+u(p1-p0)+v(p2-p0)}$

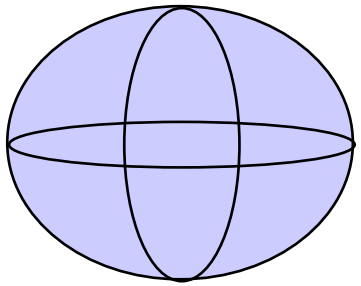


2.Parametric Sphere参数球面

$x(\theta, \phi) = r \cos\theta \sin\phi$
$y(\theta, \phi) = r \sin\theta \sin\phi$
$z(\theta, \phi) = r \cos\phi$

$360 \geq \theta \geq 0$
$180 \geq \phi \geq 0$

θ constant: circles of constant longitude
ϕ constant: circles of constant latitude



Solid Modeling /3.B-reps /Spline样条

$$x(u)=\sum_{i=0}^{N} c_{xi}u^i \qquad y(u)=\sum_{j=0}^{M} c_{yj}u^j \qquad z(u)=\sum_{k=0}^{L} c_{zk}u^k$$

$$\mathrm{p}(u)=\sum_{k=0}^{L} c_k u^k$$

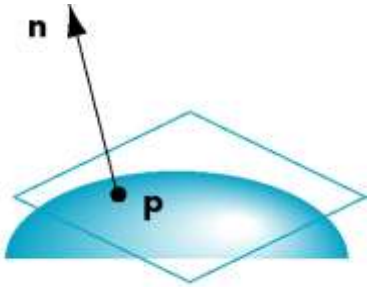
- Easy to evaluate (求值)
  - After normalizing u（规范化U）,each curve is written **p**(u)=[x(u), y(u), z(u)]^T, （$1 \geq u \geq 0$）
  - After normalizing u（规范化U）,surface is written p(u,v) = [x(u,v), y(u,v), z(u,v)] (0<=u<=1, 0<=v<=1)
- Continuous（连续）and differentiable（微分）everywhere

- easy to differentiate（易于求导，求法向量）
  - differentiate with respect to u and v to obtain the normal at any point **p**

$$\frac{\partial \mathbf{p}(u,v)}{\partial u}=\begin{bmatrix}\partial \mathrm{x}(u,v)/\partial u\\ \partial \mathrm{y}(u,v)/\partial u\\ \partial \mathrm{z}(u,v)/\partial u\end{bmatrix}$$

$$\mathbf{n}=\frac{\partial \mathbf{p}(u,v)}{\partial u}\times\frac{\partial \mathbf{p}(u,v)}{\partial v}$$

$$\frac{\partial \mathbf{p}(u,v)}{\partial v}=\begin{bmatrix}\partial \mathrm{x}(u,v)/\partial v\\ \partial \mathrm{y}(u,v)/\partial v\\ \partial \mathrm{z}(u,v)/\partial v\end{bmatrix}$$



Solid Modeling /3.B-reps /Spline样条（cont.）

- **PC Curve：Cubic Parametric Polynomials（三次参数多项式表示）**

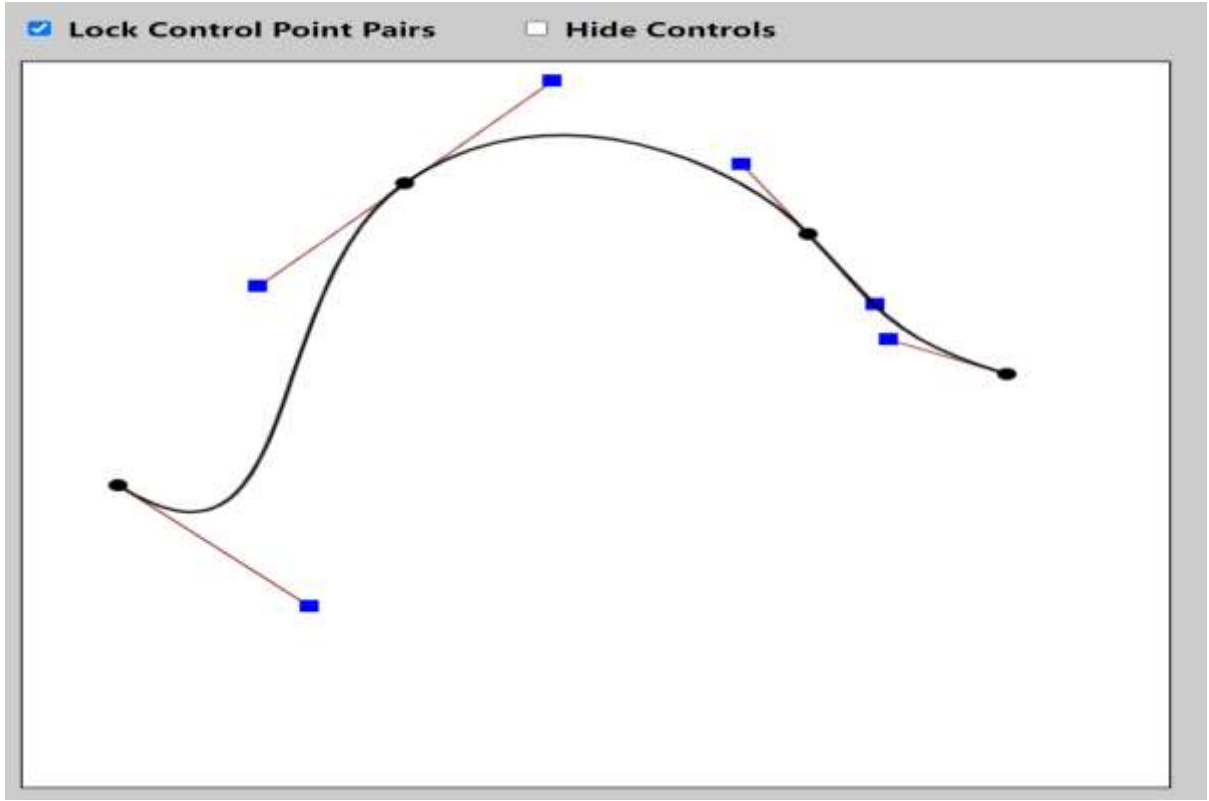
$$p(u) = \sum_{k=0}^{3} c_k u^k \qquad p(u,v) = \sum_{i=0}^{3}\sum_{j=0}^{3} c_{ij} u^i v^j$$

- n=3, gives balance between ease of **evaluation**（求值） and **flexibility** （灵活弹性）in design
  - 拟合曲线的次数过高：过拟合
  - 拟合曲线的次数太低：欠拟合

Solid Modeling /3.B-reps /Spline样条（cont.）

- **Segments and Continuity（分段与连续）**

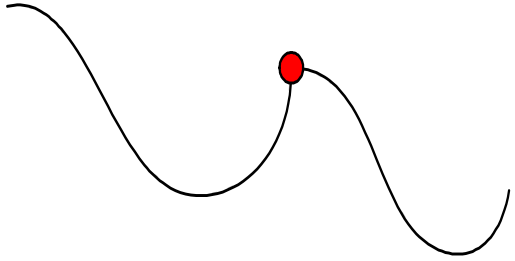
  Bezier Curve Demos: https://math.hws.edu/eck/cs424/notes2013/canvas/bezier.html



Solid Modeling /3.B-reps /Spline样条（cont.）

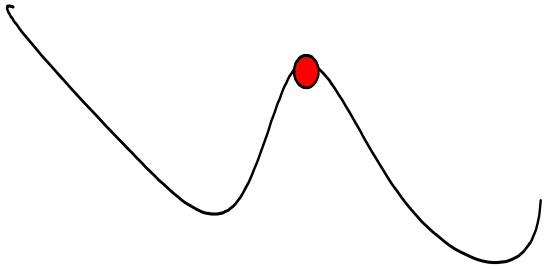
➢Segments and Continuity(cont.)

- 0阶参数连续性the parametric continuity C0：两个相邻曲线段端点处重合。
- 1阶参数连续性 the parametric continuity C1：两个相邻曲线段交点处重合，并具有相同的一阶导数。一阶连续性对数字化绘图及一些设计应用已经足够。
- 2阶参数连续性 the parametric continuity C2：两个相邻曲线段交点处重合，并具有相同的一阶和二阶导数。二阶连续性对电影中的动画路径和精密CAD需求有用。



(a) 0阶连续性

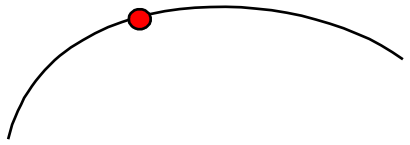
$$p_i(t_{i1}) = p_{(i+1)}(t_{(i+1)0})$$



(b) 1阶连续性

$$p_i(t_{i1}) = p_{(i+1)}(t_{(i+1)0})$$

且 $p_i'(t_{i1}) = p'_{(i+1)}(t_{(i+1)0})$



(c) 2阶连续性

$$p_i(t_{i1}) = p_{(i+1)}(t_{(i+1)0})$$

且 $p_i'(t_{i1}) = p'_{(i+1)}(t_{(i+1)0})$

且 $p_i''(t_{i1}) = p''_{(i+1)}(t_{(i+1)0})$

Solid Modeling /3.B-reps /Spline样条（cont.)

➢Segments and Continuity(cont.)

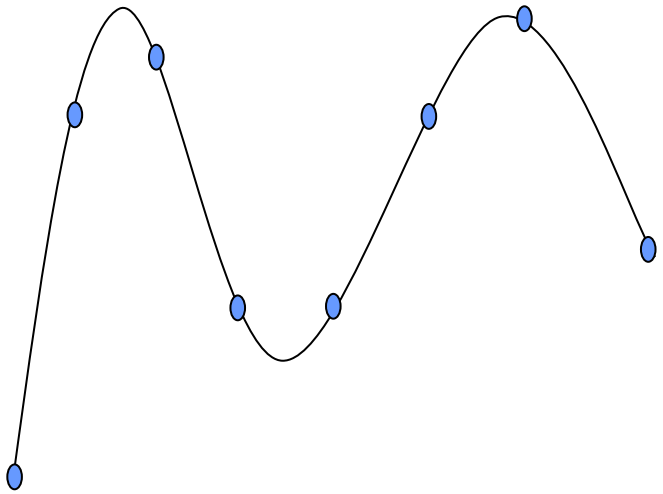
- **0阶几何连续性**，G^0
  - 指相邻曲线段在连接点处位置重合。等价于C^0
- **1阶几何连续性**，G^1
  - 指相邻曲线段在连接点处位置重合，一阶成比例（即方向相同，大小不同）
- **2阶几何连续性**，G^2
  - 指相邻曲线段在连接点处位置重合，一阶和二阶导数均成比例。

➢**C^n连续保证G^n连续，但反之不然。**

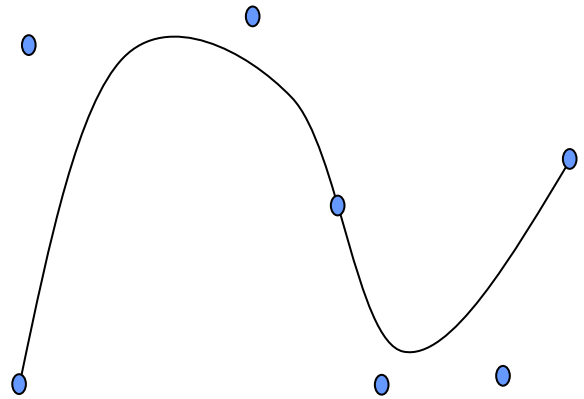
➢**C^n连续的条件比G^n连续的条件要苛刻**

Solid Modeling /3.B-reps /Spline样条(cont.)

- 构造样条曲线的方法：
  - fitting拟合：一组给定的点，拟合出某函数（模型）表示该曲线



interpolating（插值） data point
（构造的曲线过已知插值点）



approximating（逼近） data point
（构造的曲线逼近控制点）

Solid Modeling /3.B-reps /Spline样条（cont.)

➢构造样条曲线~插值拟合

1）线性插值： 已知P0(x1,y1)和P1(x2,y2)是已知线段的端点,带入参数方程,求得系数矩阵A(A_1 A_0),
则该拟合曲线表示为：

$$f(t) =(1-t)P_0+tP_1 = (P_1-P_0)\,t+P_0 = A_1t+A_0,\quad t\in[0,1]$$

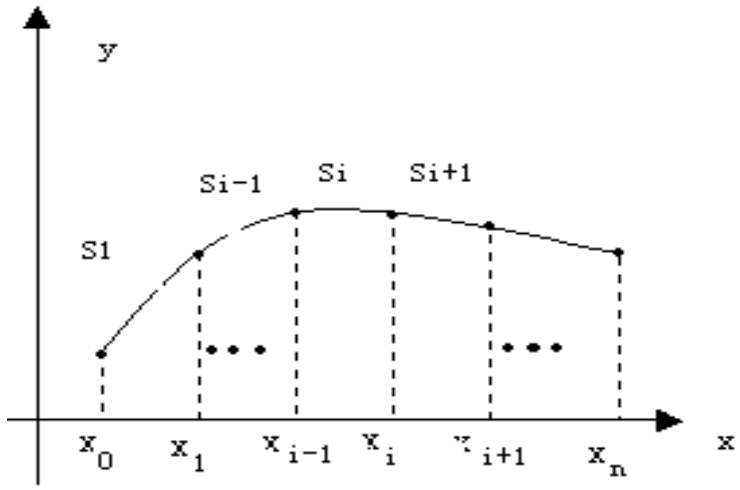
2)抛物线插值（二次插值）： 已知曲线在三个互异点（型值点）有三个值(x1, y1), (x2,y2), (x3,y3), 带入二次参数方程，求得系数矩阵A (A_2 A_1 A_0),
则该插值曲线可表示为：

$$f(t)=A_2t^2+A_1t+A_0,\quad t\in[0,1]$$

Solid Modeling /3.B-reps /Spline样条(cont.)

1)Natural Cubic Spline自然三次样条(插值法):

整个曲线表示为N段，**在所有曲线段的连接处均具有零阶、一阶和二阶导数的连续性，曲线有C^2连续性。**



N+1个型值点，N段样条曲线段，每段是M次多项式:

1)总共待求系数　(m+1)*n=4n　(m=3)

2)为解出系数，找方程:

(1)S(x)经过这n+1个有序点，可建立n+1个方程

(2)根据连续性条件，在整个区间[a,b]上，Si(t)有0、1、2 阶连续性，可建立(n-1)*3个方程.

3）添加2个补齐条件

$4n-((n+1)+(n-1)*3)=2$

- 方法之一是在P_0和P_n处设其二阶导数为0。
- 方法之二是增加二个“隐含”控制点，各自位于控制点序列的两端。即增加P_{-1}和P_{n+1}控制点，这时，所有的控制点都成为了内点 。

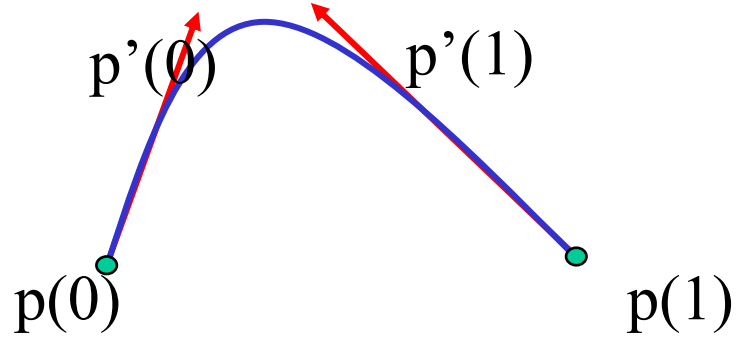
缺点：“全局控制”：如果控制点中的任何一个改动，则整个曲线都受影响。即不允许“局部控制”

Solid Modeling /3.B-reps /Spline样条（cont.）

2) Cubic Hermite Spline三次埃尔米特样条

$$P(t)= TA=A_3t^3+A_2t^2+A_1t+A_0$$

$$P'(t)=2tA_3+tA_2+A_1$$



已知：矢量P_k、P_{k+1}、R_k和R_{k+1}，

$$p(0)=P_k,\ p(1)=P_{k+1}$$
$$p'(0)=R_k,\ p'(1)=R_{k+1}$$

$P_k=P(0)=A_0$

$P_{k+1}=P(1)=A_3+A_2+A_1+A_0$

$R_k=P'(0)=A_1$

$R_{k+1}=P'(1)=2A_3+A_2+A_1$

$A3=2P_k-2P_{k+1}+R_k+R_{k+1}$

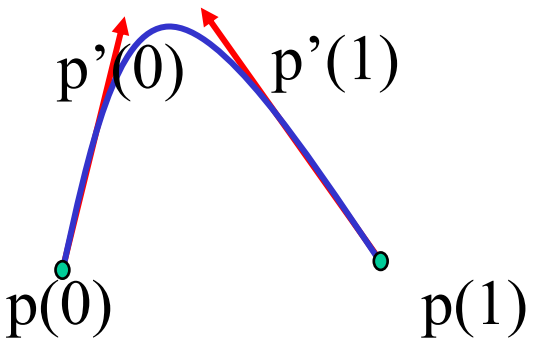
$A2=-3P_k+3P_{k+1}-2R_k-R_{k+1}$

$A1=R_{k+1}$

$A0=P_k$

Solid Modeling /3.B-reps /Spline样条（cont.）

- **Cubic Hermite Spline三次埃尔米特插值样条曲线的代数表达 P(t)=T(t) A**



$$\mathbf{A}=\begin{matrix}A3\\A2\\A1\\A0\end{matrix}=\begin{bmatrix}2&-2&1&1\\-3&3&-2&-1\\0&0&1&0\\1&0&0&0\end{bmatrix}\begin{bmatrix}P_k\\P_{k+1}\\R_k\\R_{k+1}\end{bmatrix}=M_h\times G_h$$

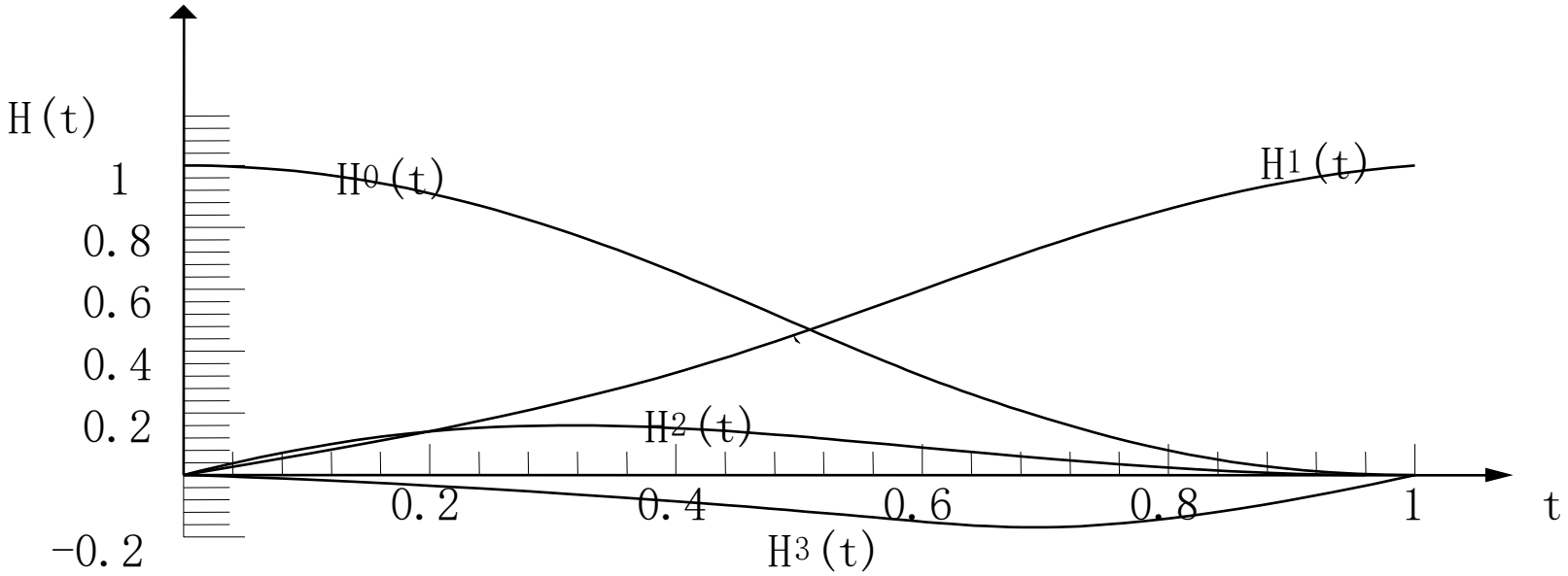
$$P(t)=\begin{bmatrix}t^3&t^2&t&1\end{bmatrix}\begin{bmatrix}2&-2&1&1\\-3&3&-2&-1\\0&0&1&0\\1&0&0&0\end{bmatrix}\begin{bmatrix}P_k\\P_{k+1}\\R_k\\R_{k+1}\end{bmatrix}$$

Solid Modeling /3.B-reps /Spline样条（cont.）

- **Cubic Hermite Spline三次埃尔米特样条曲线的几何表达 P(t)=H(t) G**
  - **Gh 称为几何系数，** H(t)称为**Hermite基函数/调和函数**
  - 三次Hermite插值样条曲线包含四项求和：

$$p(t) = P_k H_0(t) + P_{k+1} H_1(t) + R_k H_2(t) + R_{k+1} H_3(t)$$

$$G_h = \begin{bmatrix} P_k \\ P_{k+1} \\ R_k \\ R_{k+1} \end{bmatrix}$$



$$H(t) = \begin{bmatrix} H_0(t) \\ H_1(t) \\ H_2(t) \\ H_3(t) \end{bmatrix} = \begin{bmatrix} 2t^3 - 3t^2 + 1 \\ -2t^3 + 3t^2 \\ t^3 - 2t^2 + t \\ t^3 - t^2 \end{bmatrix}$$

图　Hermite blend function（基函数/调和函数）

Solid Modeling /3.B-reps /Spline样条（cont.)

➢3.Cardinal Spline

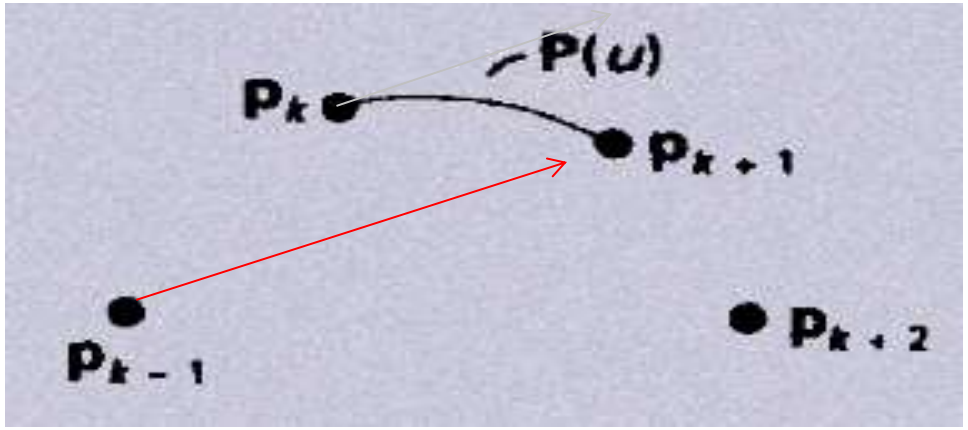
- Cardinal样条段的边界条件（导数）可由其两个相邻控制点坐标计算得到

$P(0)=P_k$

$P(1)=P_{k+1}$

$P'(0)= (1-s) *(P_{k+1}-P_{k-1})/2$

$P'(1)= (1-s) *(P_{k+2}-P_k)/2$



// 注意：这里的s是可调参数，用来控制端点切导大小)

Solid Modeling /3.B-reps /Spline样条(cont.)

- **3.Cardinal Spline**
  - 求解出代数系数后，得Cardinal曲线表示

$$p(t)=\begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix}\begin{bmatrix} -s & 2-s & s-2 & s \\ 2 & (s-3)s & 3-2s & -s \\ -s & 0 & s & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}\begin{bmatrix} P_{k-1} \\ P_k \\ P_{k+1} \\ P_{k+2} \end{bmatrix}, t\in[0,1]$$

$$=T\cdot M_{be}\cdot G_{be}$$

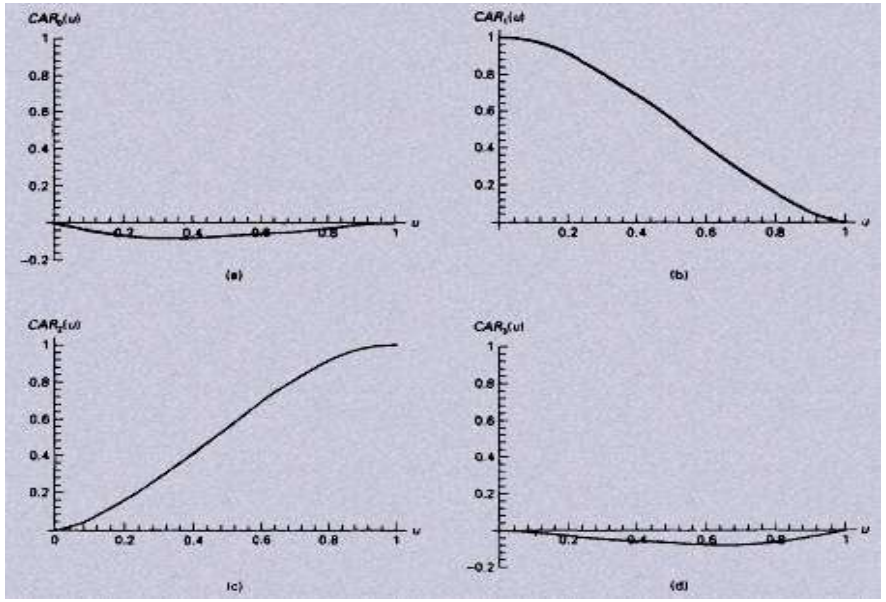
$$P(t)=P_{k-1}CAR_0(t)+P_kCAR_1(t)+P_{k+1}CAR_2(t)+P_{k+2}CAR_3(t)$$

$$CAR_0(t)=-st^3+2st^2-st$$

$$CAR_1(t)=(2-s)t^3+(s-3)st^2+1$$

$$CAR_2(t)=(s-2)t^3+(3-2s)t^2+st$$

$$CAR_3(t)=st^3-st^2$$



上图：s=0时的Cardinal混合函数

Solid Modeling /3.B-reps /Spline样条（cont.）

➢4.Catmull-Rom样条

- **Cardinal样条段的边界条件*P(u)的端点切导的计算***

  $P'(0)=(1-s)*(P_{k+1}-P_{k-1})/2;$

  $P'(1)=(1-s)*(P_{k+2}-P_{k})/2;$

- **Catmull-Rom 样条是 Cardinal 样条的一个特例，张力参数 s= 0**

与基数样条的关系：

基数样条有一个通用的张力参数 s（有时记为 c）。点 P_k 处的切线公式为：

$$T_k = (1-s)(P_{k+1} - P_{k-1})/2$$

- 当 $s = 0$ 时，就是 **Catmull-Rom 样条**。
- 当 $s > 0$ 时，曲线会变得更"紧"，更靠近控制点多边形。
- 当 $s < 0$ 时，曲线会变得更"松"，更像一个松弛的弹簧。

您对它的定义非常准确：**Catmull-Rom 样条本质上就是张力（Tension）参数为 0 的基数样条（Cardinal Spline）**。它是一种在图形学、动画和路径规划中非常流行的插值样条。

核心特性

Catmull-Rom 样条最吸引人的特点是：

1. **插值性**：样条曲线会**精确地穿过每一个给定的控制点**（除了首尾两个点）。这与贝塞尔曲线（Bézier Curve）不同，贝塞尔曲线只通过起点和终点，而不通过中间的控制点。
2. **局部控制性**：移动任意一个控制点，只会影响该点邻近的曲线段，而不会影响整个曲线。这使调整变得非常方便。
3. **C1 连续性**：曲线在控制点处是**一阶连续（切线连续）**的，这意味着整个曲线看起来非常平滑，没有突兀的尖角。
4. **无需指定切线**：曲线的切线方向由相邻的控制点自动计算得出，用户只需要提供位置点即可。

Solid Modeling /3.B-reps /Spline样条（cont.）

➢2. 逼近（Approximation）拟合

- ➢选择一个次数较低的函数,在某种意义上最佳逼近给定点(控制点).
- ➢该函数不必通过所有控制点。

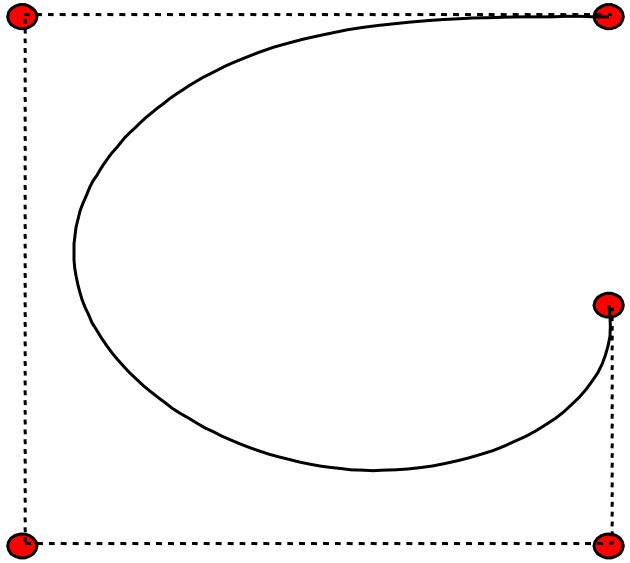
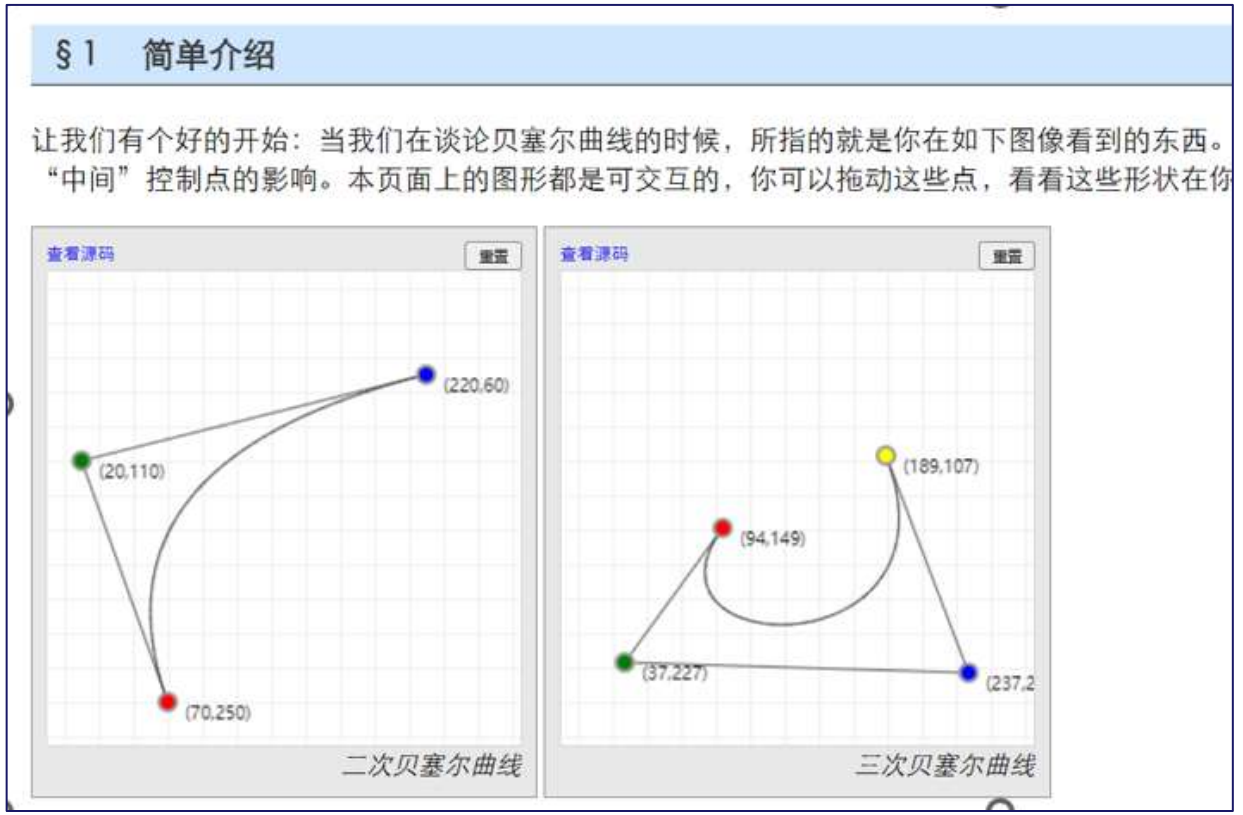


图　逼近的拟合曲线

Solid Modeling /3.B-reps /Spline样条（cont.）

- 逼近样条举例：Bezier Curve贝塞尔曲线
  - 常用的采用Approximation（逼近）方法构造的样条曲线
  - *参“贝塞尔曲线入门”https://pomax.github.io/bezierinfo/zh-CN/index.html#matrixsplit*





Solid Modeling /3.B-reps /Spline样条（cont.）

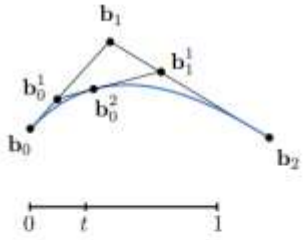
➢Bezier Curve贝塞尔曲线（cont.）

3) Cubic Bezier curve –De Casteljau Algorithm (cont.)

- **根据如下的三个控制点的递归计算过程，推导出来的就是贝塞尔公式！**
  - *参见GAMES101 Lecture11 geometry2*

Bézier Curve – Algebraic Formula

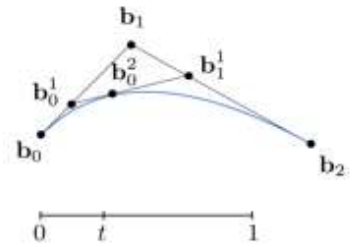
Example: quadratic Bézier curve from three points



$$\mathbf{b}_0^1(t) = (1-t)\mathbf{b}_0 + t\mathbf{b}_1$$
$$\mathbf{b}_1^1(t) = (1-t)\mathbf{b}_1 + t\mathbf{b}_2$$
$$\mathbf{b}_0^2(t) = (1-t)\mathbf{b}_0^1 + t\mathbf{b}_1^1$$

$$\boxed{\mathbf{b}_0^2(t) = (1-t)^2\mathbf{b}_0 + 2t(1-t)\mathbf{b}_1 + t^2\mathbf{b}_2}$$

Run the same algorithm for every t in [0,1]



$$p(t) = \sum_{k=0}^{2} P_k BEN_{k,2}(t)$$
$$= \boxed{(1-t)^2 P_0 + 2t(1-t)P_1 + t^2 P_2} \qquad t \in [0,1]$$
$$= (P_2 - 2P_1 + P_0)t^2 + 2(P_1 - P_0)t + P_0$$

Solid Modeling /3.B-reps /Spline样条（cont.)

➢ Bezier Curve贝塞尔曲线（cont.)

Bezier曲线: 用一组基函数,对一组已知控制点进行线性组合的结果

$$p(t) = \sum_{k=0}^{n} P_k BEN_{k,n}(t) \qquad t \in [0,1]$$



Bernstein基函数BEN(t)具有如下形式：

$$BEN_{k,n}(t) = \frac{n!}{k!(n-k)!} t^k (1-t)^{n-k} = C_n^k t^k (1-t)^{n-k}$$

$$k = 0, 1, \cdots, n$$



$$C_n^k = \frac{n!}{k!(n-k)!} = \frac{n-k+1}{n} C_n^{k-1} \qquad n \geq k$$

注意：当k=0，t=0时，t^k=1，k!=1(即0^0=1,0!=1)

Solid Modeling /3.B-reps /Spline样条（cont.）

➢Cubic Bezier Curve三次贝塞尔曲线

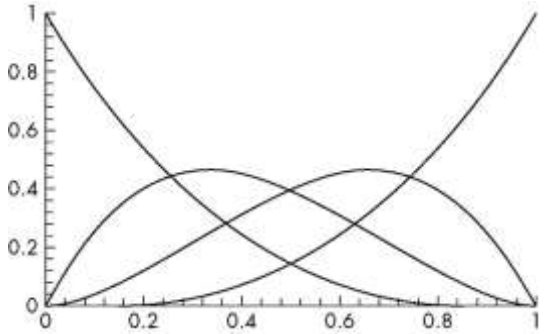
$$p(t)=\sum_{k=0}^{3} P_k BEN_{k,3}(t)$$

$$=(1-t)^3 P_0+3t(1-t)^2 P_1+3t^2(1-t)P_2+t^3 P_3 \qquad t\in[0,1]$$

$$p(t)=\begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix}\begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}\begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix} \qquad t\in[0,1]$$

$$=T\cdot M_{be}\cdot G_{be}$$

$$\mathbf{b}(u)=\begin{bmatrix} (1-u)^3 \\ 3u(1-u)^2 \\ 3u^2(1-u) \\ u^3 \end{bmatrix}$$



Note that all zeros are at 0 and 1 which forces the functions to be smooth over (0,1)

Solid Modeling /3.B-reps /Spline样条（cont.)

➢贝塞尔曲线的性质

1. 端点在曲线上

$$p(0)=\sum_{k=0}^{n}P_k BEN_{k,n}(0)$$

$$=P_0 BEN_{0,n}(0)+P_1 BEN_{1,n}(0)+\cdots+P_n BEN_{n,n}(0)$$

$$=P_0$$

$$p(1)=\sum_{k=0}^{n}P_k BEN_{k,n}(1)$$

$$=P_0 BEN_{0,n}(1)+P_1 BEN_{1,n}(1)+\cdots+P_n BEN_{n,n}(1)$$

$$=P_n$$

Solid Modeling /3.B-reps /Spline样条(cont.)

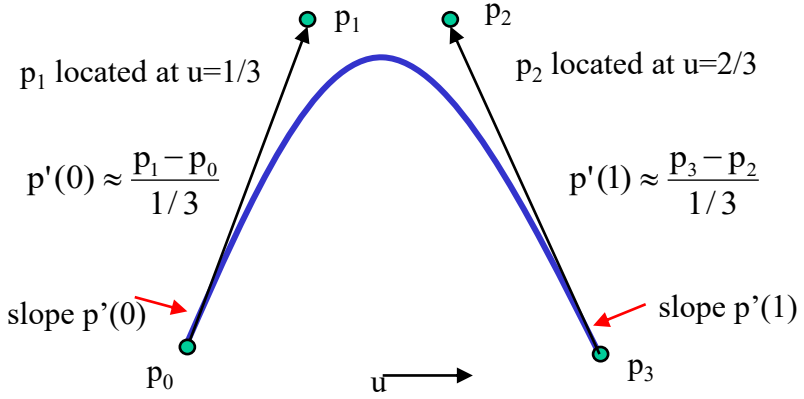
- 贝塞尔曲线的性质

2. 端点一阶导数由控制点计算得到

$$p'(t) = n\sum_{k=0}^{n} P_k (BEN_{k-1,n-1}(t) - BEN_{k,n-1}(t))$$
$$= n((P_1 - P_0)BEN_{0,n-1}(t) + (P_2 - P_1)BEN_{1,n-1}(t)$$
$$+ \cdots + (P_n - P_{n-1})BEN_{n-1,n-1}(t))$$
$$= n\sum_{k=1}^{n} (P_k - P_{k-1})BEN_{k-1,n-1}(t)$$

$$p'(0) = n(P_1 - P_0)$$
$$p'(1) = n(P_n - P_{n-1})$$

$$p'(0) = 3(P_1 - P_0)$$
$$p'(1) = 3(P_3 - P_2)$$



Solid Modeling /3.B-reps /Spline样条（cont.）

- 贝塞尔曲线的性质

3. Bezier曲线段端点的二阶导数易于计算

$$p''(0) = n(n-1)((P_2 - P_1) - (P_1 - P_0))$$
$$p''(1) = n(n-1)((P_{n-2} - P_{n-1}) - (P_{n-1} - P_n))$$

三次Bezier曲线段端点的二阶导数：

$$p''(0) = 6(P_0 - 2P_1 + P_2)$$
$$p''(1) = 6(P_1 - 2P_2 + P_3)$$

Solid Modeling /3.B-reps /Spline样条(cont.)

- 贝塞尔曲线的性质

4. 对称性：$B_{i,n}(t)=B_{n-i,n}(1-t)$，即基函数具有对称性。

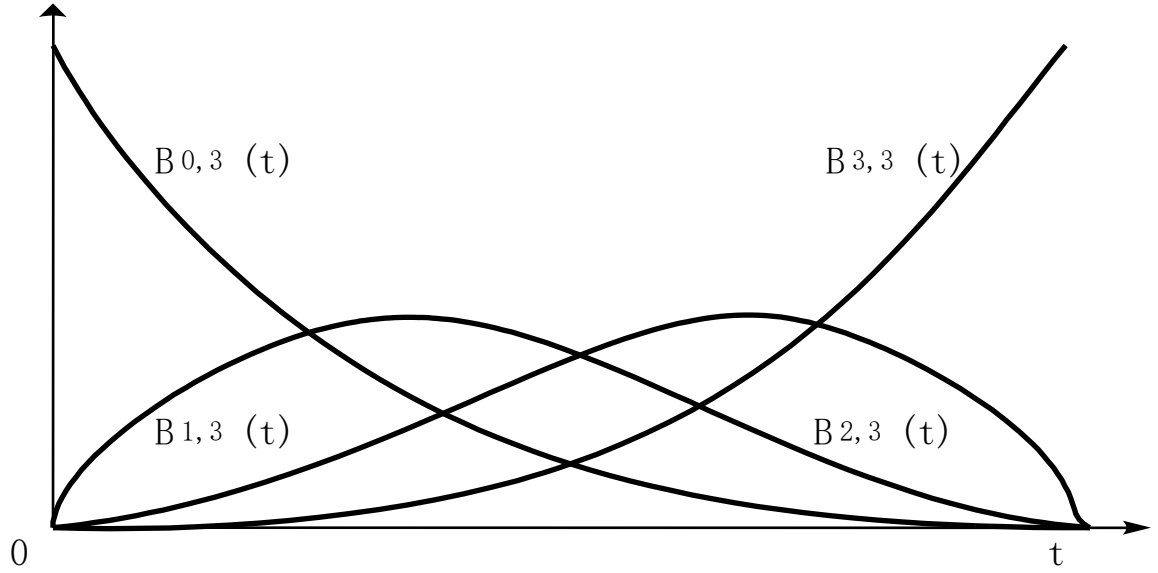


图　三次Bezier曲线四个Bezier基函数(调和函数)

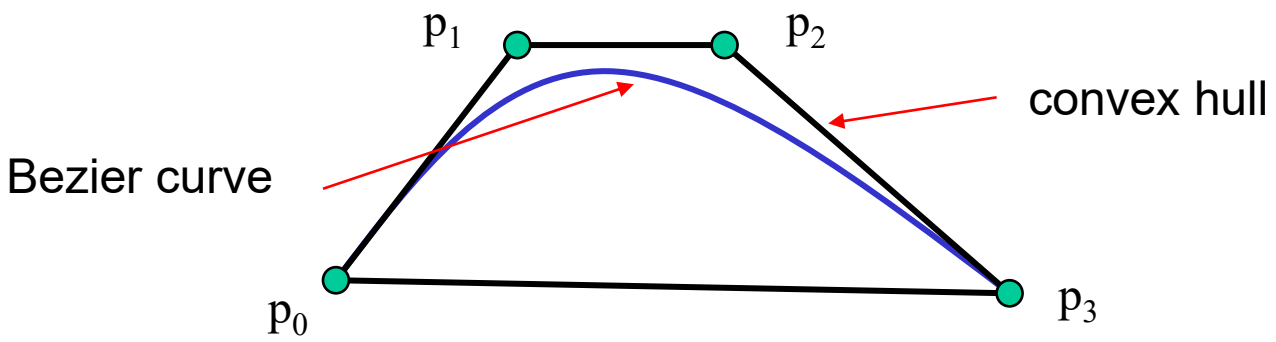
Solid Modeling /3.B-reps /Spline样条（cont.)

- 贝塞尔曲线的性质

5. Convex hull 凸包性(*)

$$BEN_{k,n}(t) = \frac{n!}{k!(n-k)!} t^k (1-t)^{n-k} \geq 0$$

$$\sum_{k=0}^{n} BEN_{k,n}(t) = \sum_{k=0}^{n} \frac{n!}{k!(n-k)!} t^k (1-t)^{n-k} = ((1-t)+t)^n \equiv 1$$

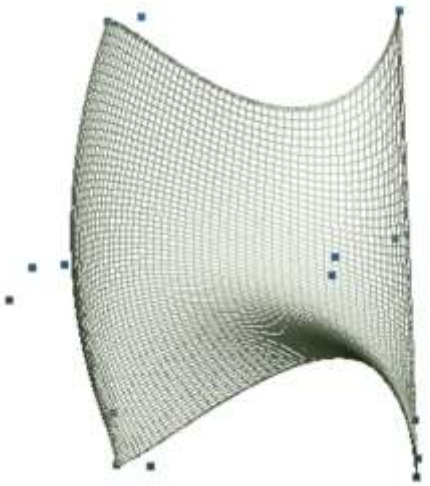


Solid Modeling /3.B-reps /Spline样条（cont.)

➢Bezier Surface贝塞尔曲面

$$p(u,v)=\sum_{i=0}^{m}\sum_{j=0}^{n}P_{i,j}BEN_{i,m}(u)BEN_{j,n}(v)$$

$$(u,v)\in[0,1]\times[0,1]$$



BENi,m(t)与BENj,n(v)是**Bernstein基函数：**

$$BEN_{i,m}(u)=C_m^i\cdot u^i\cdot(1-u)^{m-i}$$

$$BEN_{j,n}(v)=C_n^j\cdot v^j\cdot(1-v)^{n-j}$$

Solid Modeling /3.B-reps /Spline样条（cont.)

➢Double Cubic Bezier Surface双三次贝塞尔曲面

- 双三次Bezier曲面(*m*=*n*=3)，（m+1)(n+1)=16个控制点

  Using same data array $\mathbf{P}=[p_{ij}]$ as with interpolating form

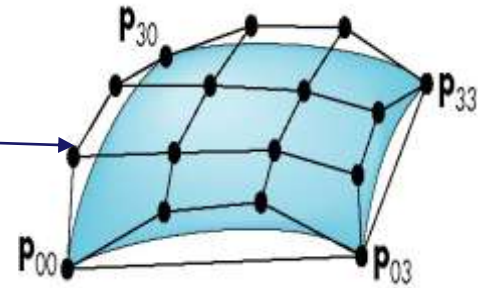
$$p(u,v)=\sum_{i=0}^{3}\sum_{j=0}^{3}P_{i,j}BEN_{i,3}(u)BEN_{j,3}(v)$$

$$p(u,v)=\sum_{i=0}^{3}\sum_{j=0}^{3}b_i(u)b_j(v)\,p_{ij}=u^T\mathbf{M}_B\mathbf{P}\mathbf{M}_B^T v$$

$$(u,v)\in[0,1]\times[0,1]$$

$$U=\begin{bmatrix}u^3 & u^2 & u & 1\end{bmatrix}\quad V=\begin{bmatrix}v^3 & v^2 & v & 1\end{bmatrix}$$

Patch lies in convex hull



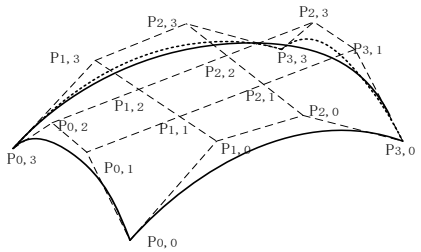


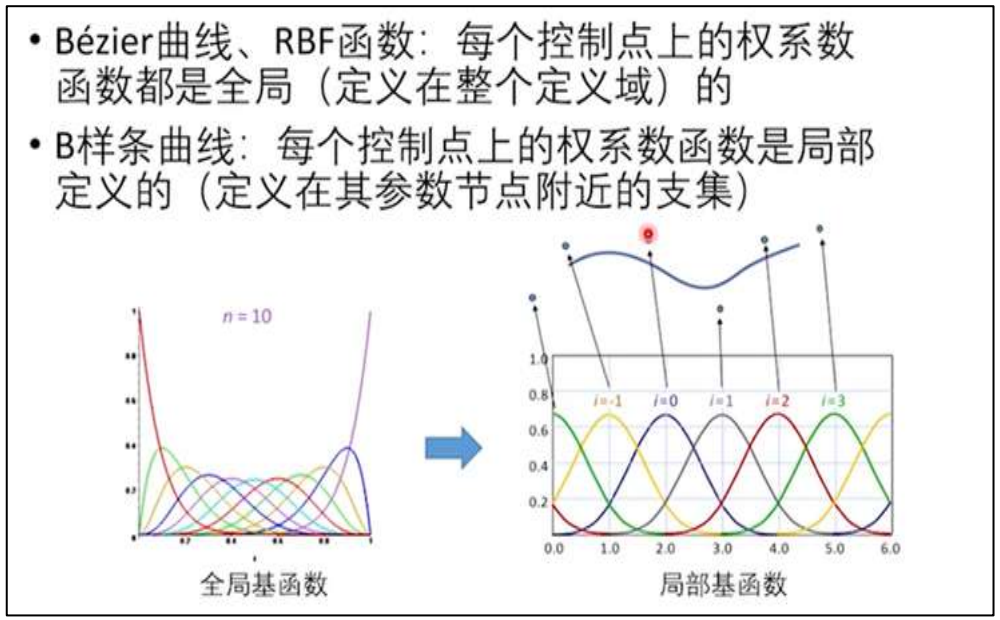
图 双三次Bezier曲面及其控制网格

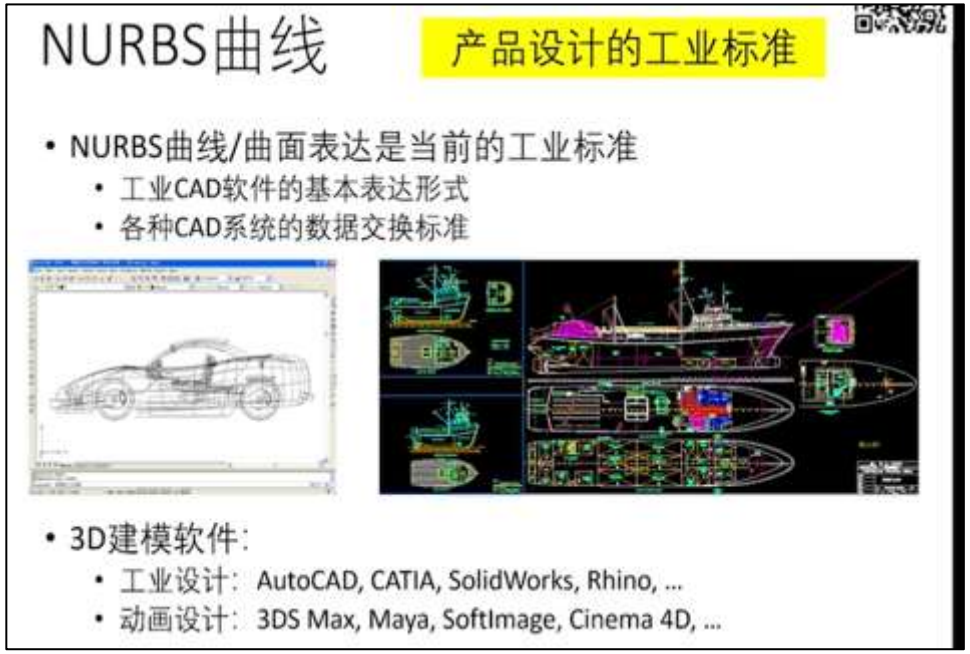
$$M_{be}=\begin{bmatrix}-1 & 3 & -3 & 1\\ 3 & -6 & 3 & 0\\ -3 & 3 & 0 & 0\\ 1 & 0 & 0 & 0\end{bmatrix}$$

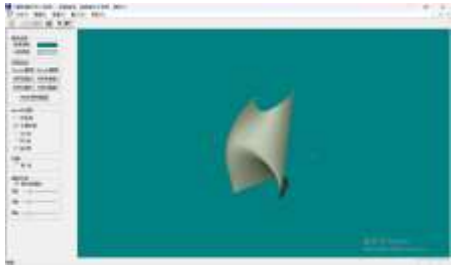
$$P=\begin{bmatrix}P_{0,0} & P_{0,1} & P_{0,2} & P_{0,3}\\ P_{1,0} & P_{1,1} & P_{1,2} & P_{1,3}\\ P_{2,0} & P_{2,1} & P_{2,2} & P_{2,3}\\ p_{3,0} & P_{3,1} & P_{3,2} & P_{3,3}\end{bmatrix}$$

Solid Modeling /3.B-reps /Spline样条（cont.）

- 其它逼近拟合曲线：B-SPLINE(B样条），NURBS(非均匀有理B样条）





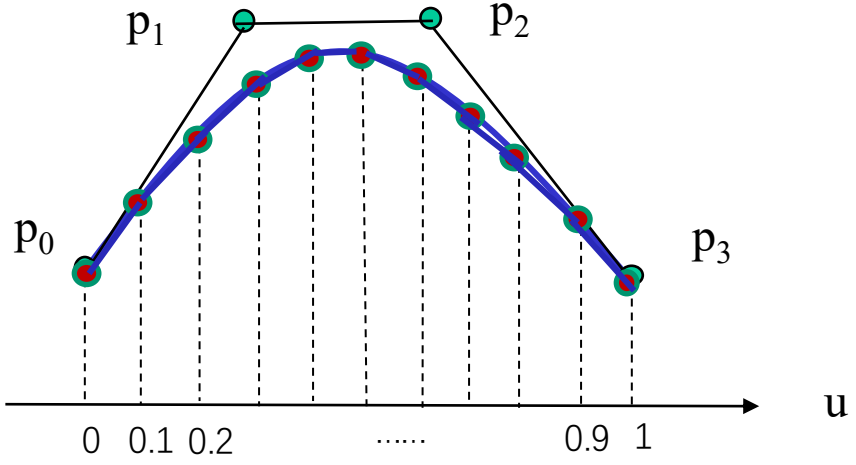


Solid Modeling /3.B-reps /Spline样条（cont.）

➢Bezier Curve Implementation Algorithm（贝塞尔曲线生成算法）

1）直接公式求值

思路：通过等间距的一组参数值计算曲线上的点，然后连接这些点，使用折线来近似表示曲线。特点：计算量大。



$$p(t)=\sum_{k=0}^{3}P_k BEN_{k,3}(t)$$

$$=(1-t)^3P_0+3t(1-t)^2P_1+3t^2(1-t)P_2+t^3P_3 \qquad t\in[0,1]$$

如：等间距划分参数区间[0,1] t=[0, 0.1，0.2，0.3,0.4,0.5,0.6,0.7,0.8,0.9,1]
对每个参数取值t，计算基函数和控制点P的点乘得曲线上的点（红点）

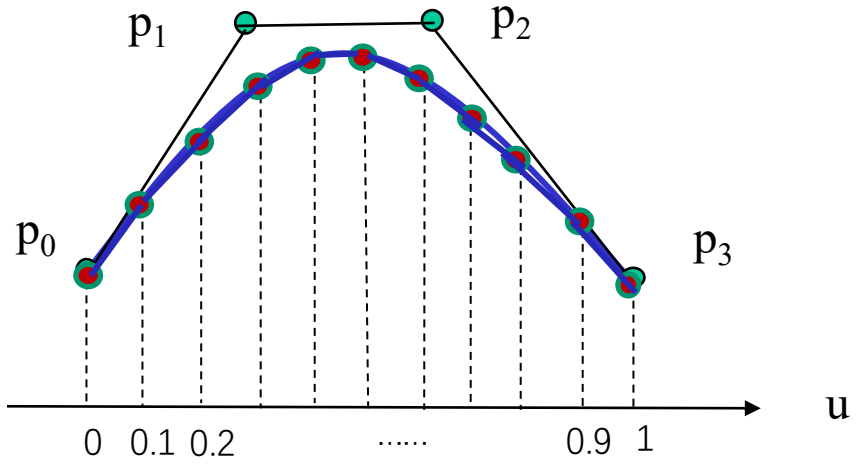
Solid Modeling /3.B-reps /Spline样条（cont.)

➢Bezier Implementation Algorithm(cont.)

2）多项式求值法polynomial evaluation（ Horner's method）

思路:通过等间距的一组参数值，根据代数公式计算曲线上的点，然后连接这些点，使用折线来近似表示曲线。

特点:计算公式有优化，但是只适合均匀网格，容易累积数值误差



$$p(u)=c_0+u(c_1+u(c_2+uc_3))$$

//3 multiplications for cubic

如：等间距划分参数区间[0,1]， t=[0, 0.1，0.2，0.3，0.4，0.5，0.6，0.7，0.8，0.9，1]
先根据公式计算得到曲线上的点（红点表示），然后每两红点之间以直代曲，绘制出曲线（绿线表示）

Solid Modeling /3.B-reps /Spline样条（cont.)

➢Bezier Implementation Algorithm(cont.)

3) Cubic Bezier curve –De Casteljau(德•卡斯特里奥） Algorithm

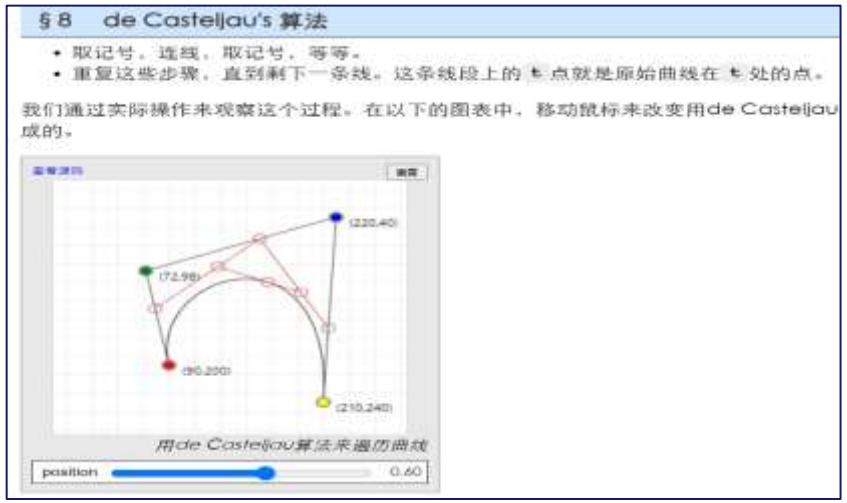
参“贝塞尔曲线入门”https://pomax.github.io/bezierinfo/zh-CN/index.html#matrixsplit

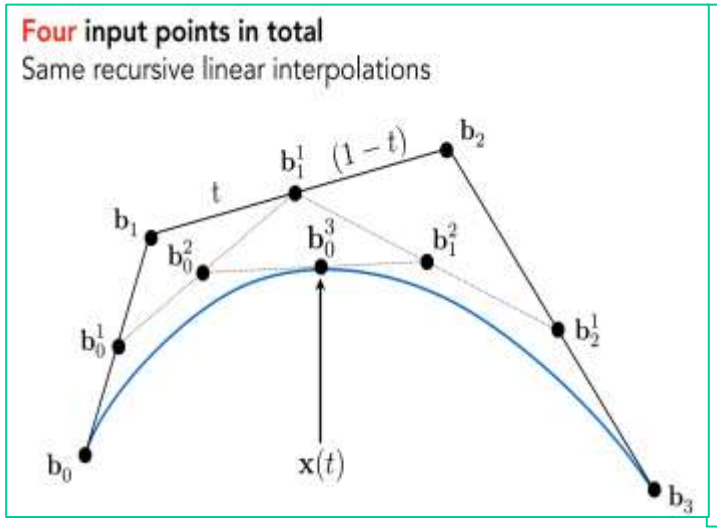


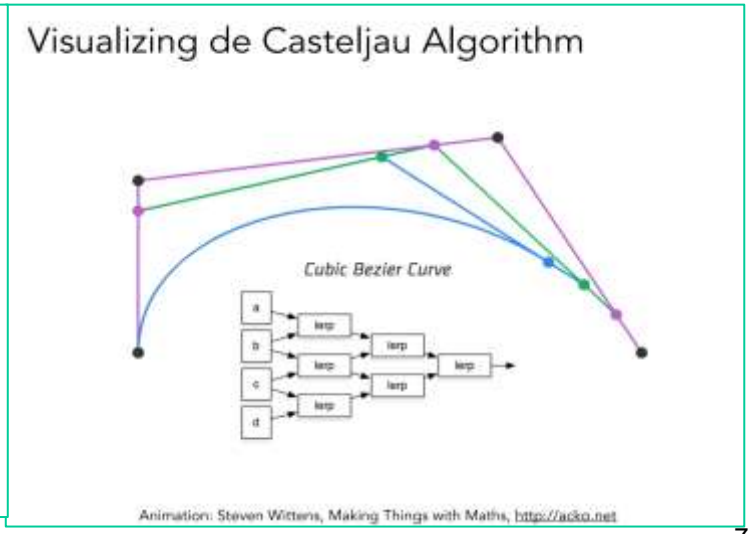
思路：通过反复对控制点做线性插值，层层“降阶”，直到只剩一个点，这个点就是曲线上

具体过程是：

- 给定控制点 $P0,P1,\ldots,Pn$
- 对相邻点做线性插值，得到一层新点集
- 对新点重复插值，点数逐层减少，
- 最终收敛到唯一点 $P(t)$







Solid Modeling /3.B-reps /Spline样条（cont.)

➢Bezier Implementation Algorithm(cont.)

4）递归细分（“分割法”）

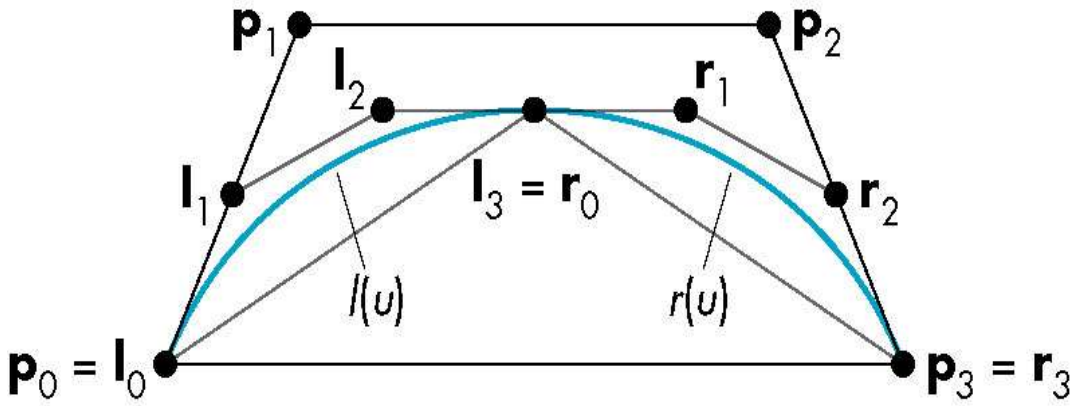
参“贝塞尔曲线入门”，https://pomax.github.io/bezierinfo/zh-CN/index.html#matrixsplit

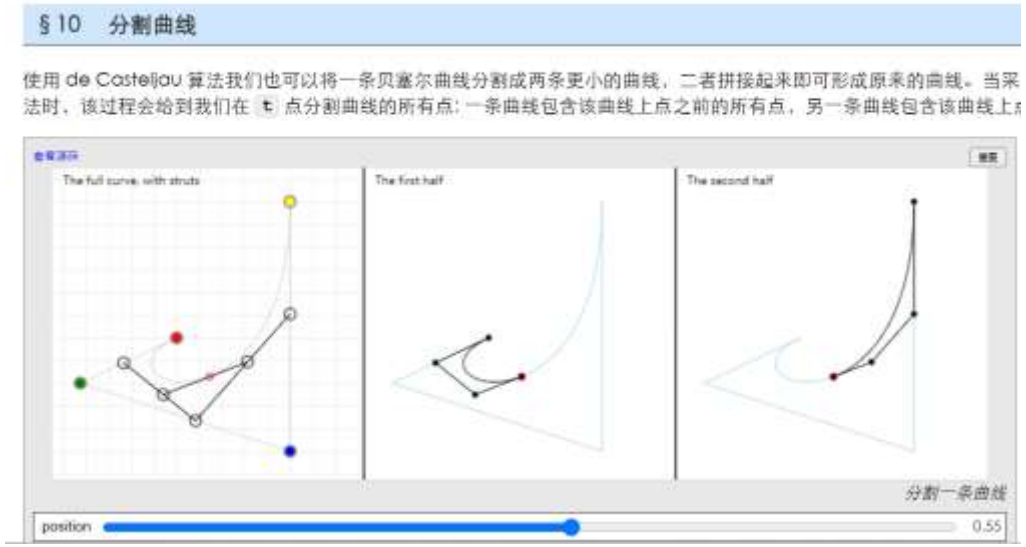
特点：use the convex hull（凸包）property of Bezier curves to obtain **an efficient recursive method**

原理：变差缩减性the *variation diminishing property* ：将曲线P(u)分割成两个独立的多项式l(u),r(u)，l(u)和r(u)的凸包都仍将位于P（u）的凸包内。

思路：不断将曲线一分为二，重新计算所分更短两段的控制点，当小段的控制点凸包“充分靠近”曲线时，就用首末两点线段表示该曲线段。

该方法与De Casteljau Algorithm**方法的主要区别：求出的是“线段”而不是“点”**





Solid Modeling /3.B-reps /Spline样条（cont.）

➢Bezier Implementation Algorithm(cont.)

4）recursive subdivision 递归细分（“分割法”）（举例）

- 初始，将p(u)的凸包控制点(P0,P1,P2,P3)分成两组控制点（l0,l1,l2,l3）,(r0,r1,r2,r3)
- 判断左边组控制点的扁平性（l_1l_2到l_0l_3的距离）或l_0l_3足够小）

  如果距离足够小，则可用线段(l_0l_3)代替该段曲线；否则，可将控制点组分成两部分并且继续判断两个新凸包的扁平性（递归调用）
- 判断右边边组控制点的扁平性（ r_1r_2到r_0r_3的距离）或r_0r_3足够小

  如果距离足够小，则可用线段(r_0r_3)代替该段曲线；否则，可将控制点组分成两部分并且继续判断两个新凸包的扁平性（递归调用）

Start with Bezier equations $p(u)=\mathbf{u}^T M_B \mathbf{p}$

l(u) must interpolate p(0) and p(1/2)

$$l(0) = l_0 = p_0$$
$$l(1) = l_3 = p(1/2) = 1/8(p_0 + 3 p_1 + 3 p_2 + p_3)$$

Matching slopes, taking into account that l(u) and r(u) only go over half the distance as p(u)

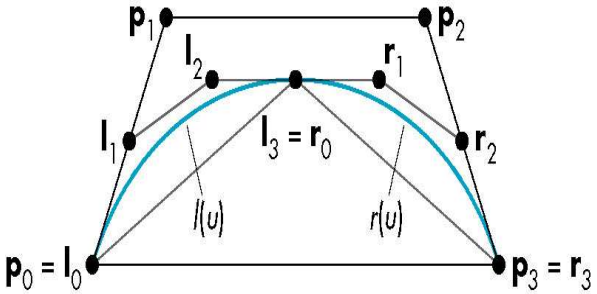
$$l'(0) = 3(l_1 - l_0) = p'(0) = 3/2(p_1 - p_0)$$
$$l'(1) = 3(l_3 - l_2) = p'(1/2) = 3/8(- p_0 - p_1 + p_2 + p_3)$$

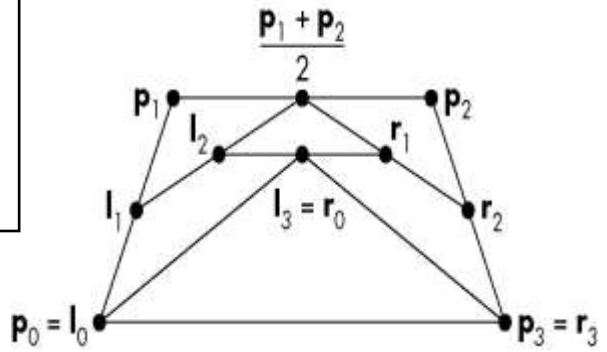
$$l_0 = p_0$$
$$r_3 = p_3$$

$$l_1 = ½(p_0 + p_1)$$
$$r_1 = ½(p_2 + p_3)$$

$$l_2 = ½(l_1 + ½(p_1 + p_2))$$
$$r_1 = ½(r_2 + ½(p_1 + p_2))$$

$$l_3 = r_0 = ½(l_2 + r_1)$$





- 四个方程联立求解可得到l(u)的控制点（l0,l1,l2,l3）的计算公式，
- 同理，对称得到r（u）的控制点值(r0,r1,r2,r3)的计算公式，

  Requires only shifts and adds!

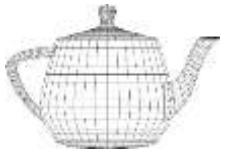
Solid Modeling /3.B-reps /Spline样条（cont.）

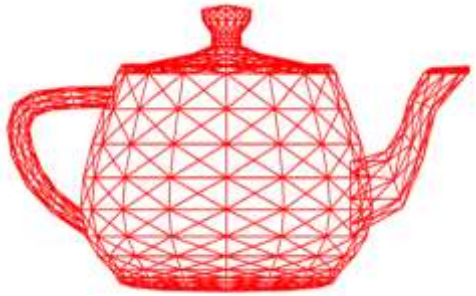
➢样条曲面实例：

➢WebGL Example: Utah Teapot 犹他茶壶（略）

- Most famous data set in computer graphics
- Widely available as a list of 306 3D vertices and the indices that define 32 Bezier patches

- ***Ref: //Angle8ECode/11/teapot.****
  - vertices.js: three versions of the data vertex data
  - patches.js: teapot patch data





  - teapot1: wire frame teapot by recursive subdivision of Bezier curves



  - teapot2: wire frame teapot using polynomial evaluation

Outline

- Properties of Geometric Objects几何对象的属性
- Geometry Objects Representation几何对象的表示
- Modeling Technology造型技术
  - 实体造型技术Solid Modeling*
    - CSG(Constructive Solid Geometry ）构造实体几何
    - SP(Space-Partitioning Representation）空间分割表示
    - B-reps(Boundary Representation)边界表示*
      - Polygon Mesh 多边形网格*
      - Spline Curve and Surface 样条曲线曲面*
  - 过程建模 process modeling
    - Fractal geometry分形几何
    - Particle system粒子系统

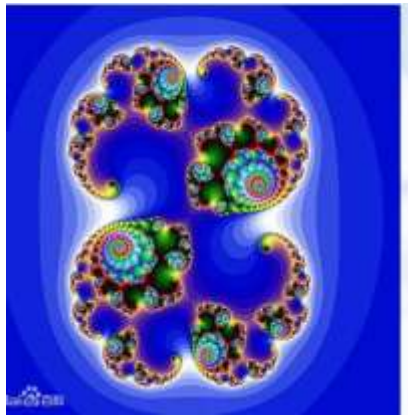
Process Modeling

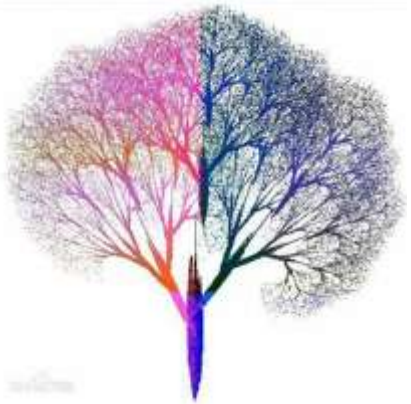
- 过程建模方法分类
  - 分形造型：
    - 利用自相似性原理，通过递归生成山峦、海岸线、云彩等不规则自然景物。
  - 粒子系统：
    - 通过控制大量“粒子”图元的生成、运动和消亡，来模拟火焰、烟花、草丛等动态且不规则的景物。
  - 基于文法的造型：
    - 最典型的是 L-系统，通过定义规则和符号来模拟树木、花草等植物的生长拓扑结构。
  - 基于物理定律或生物机制进行动态模拟
    - 连续物理模型：如基于流体力学方程模拟水流、波浪。
    - 离散模型：如细胞自动机（用于火焰模拟）、生长模型（模拟植物随环境生长）

Process Modeling- Fractal Geometry



- 著名数学家 Mandelbrot 曼德尔布诺特/芒德勃罗，于1980创立的分形几何理论
- Fractal：不规则的、破碎的、分数的，
- **分形图形是一个粗糙或零碎的几何形状，可以分成数个部分，且每一部分都（至少近似地）是 整体缩小后的形状”，即具有自相似的性质，具有以非整数维形式充填空间的形态特征。**



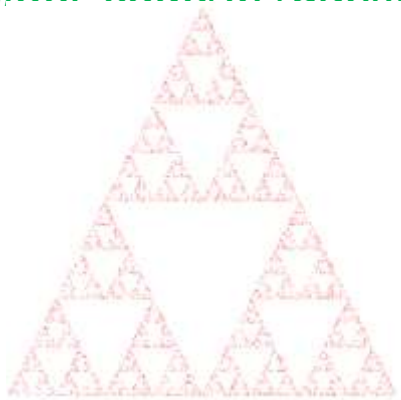


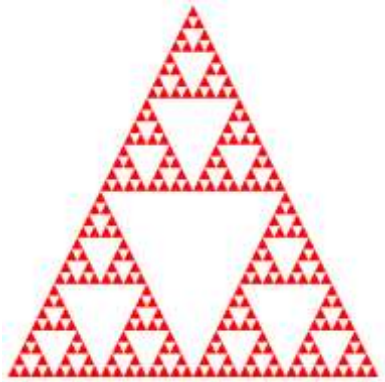


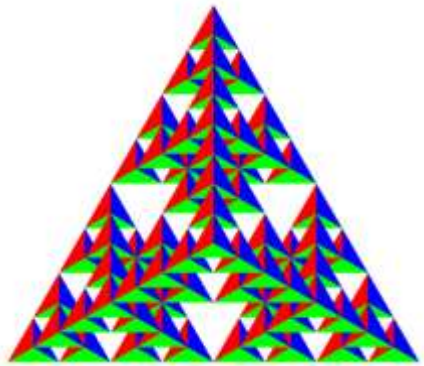


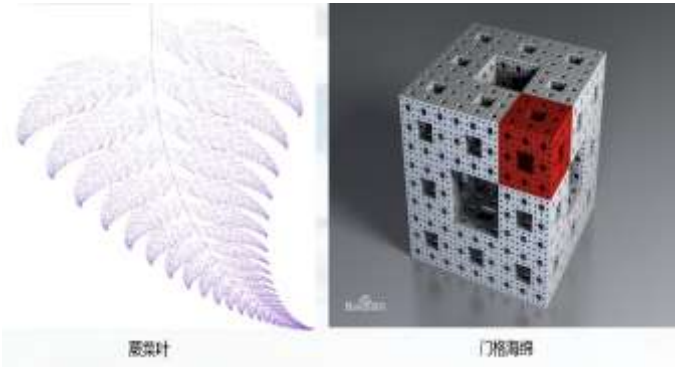
Process Modeling- Fractal Geometry（cont.)

- 分形几何(续)
  - 门格海绵，科赫雪花，龙骨曲线，…
  - *Ref: angle8Fcode/02/gasket*.html ...*











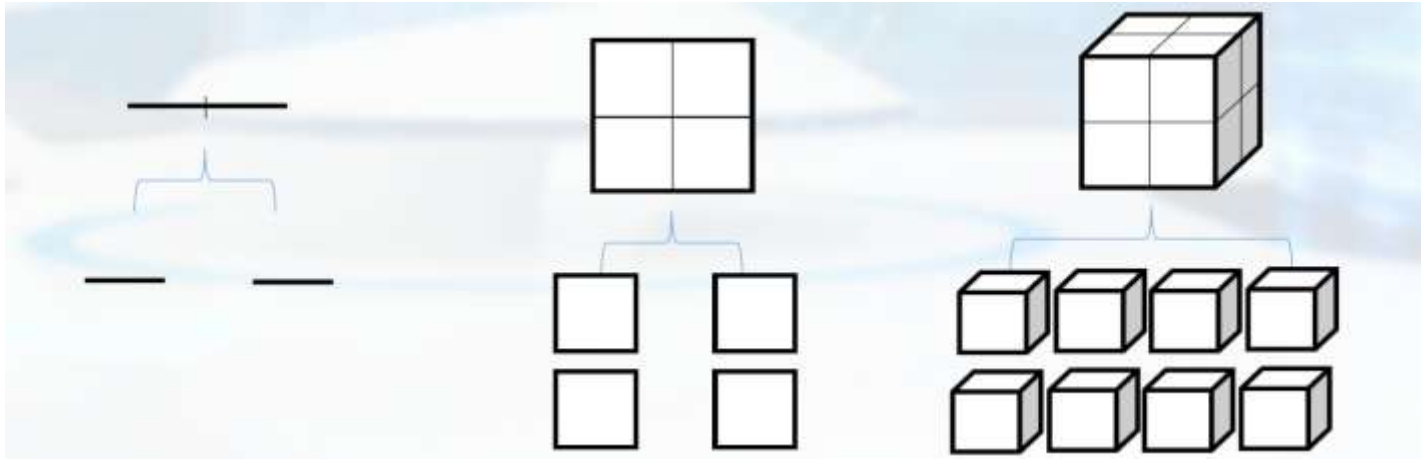


Process Modeling- Fractal Geometry（cont.)

- **分形维数/分数维数/相似性维数**
- 如果某图形是由把原图缩小为1/a的相似的b个图形所组成
  - **指数D就是分形维数，也称为相似性维数。**
  - **D可以是整数，也可以是分数（分数维）**

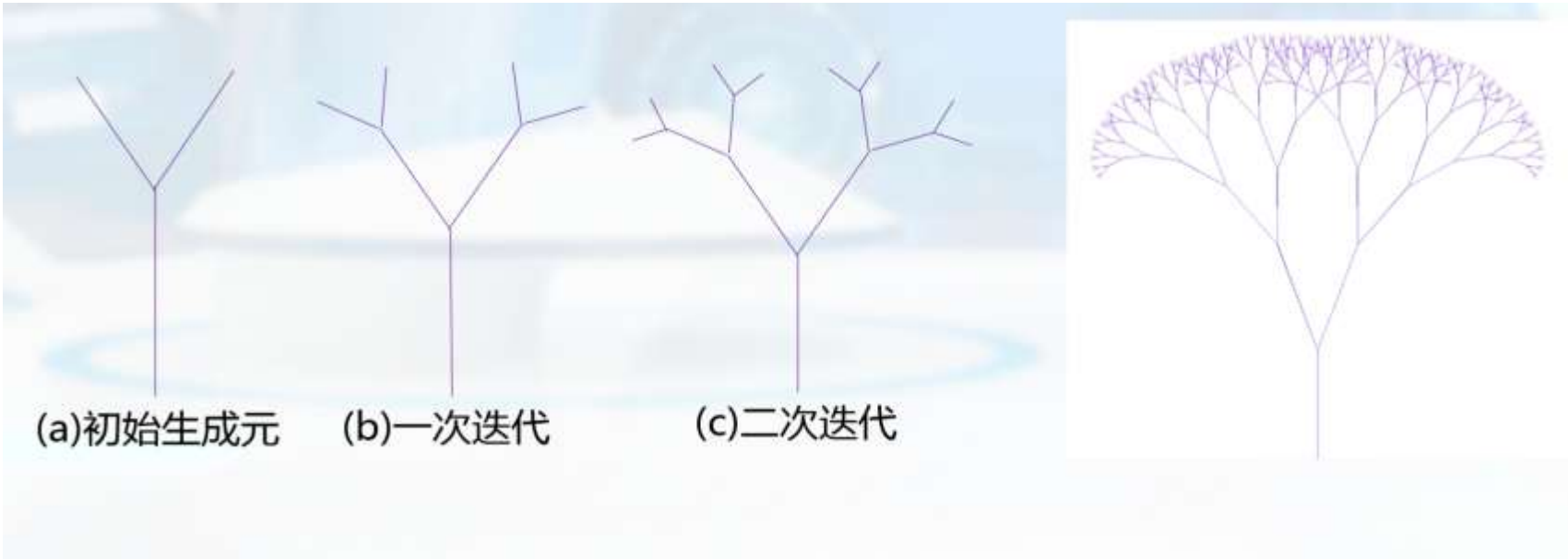
$$a^D=b$$

其线段、正方形、立方体分别被等分为2^1、2^2和2^3个相似的子图形，其中的指数1、2、3，正好等于与图形相应的经验维数。



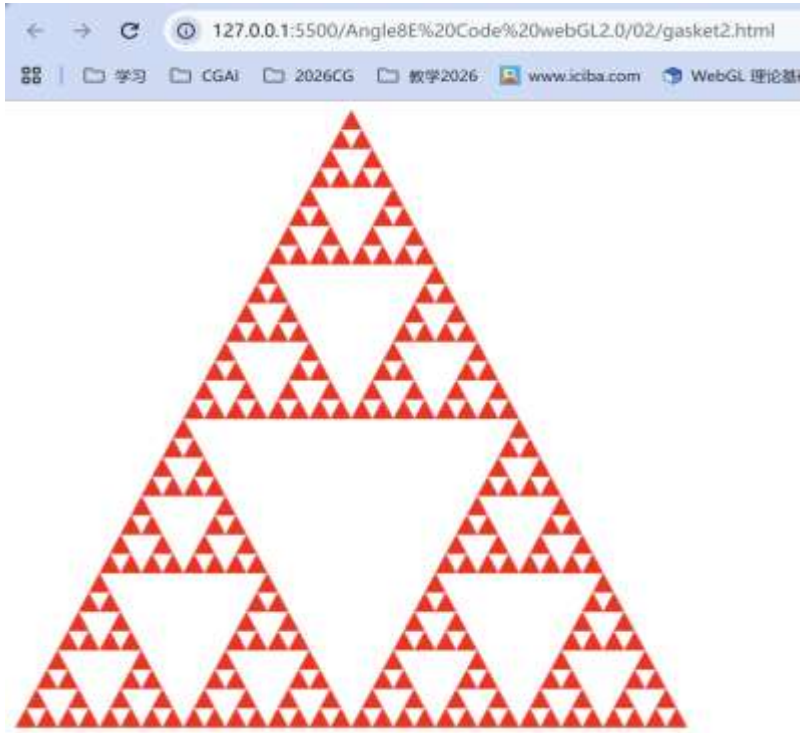
Process Modeling- Fractal Geometry（cont.)

- 分数维造型
  - 生成过程：为产生物体局部细节指定一重复操作。
  - 例：确定性自相似分形法包含两个步骤：
    - 初始生成元(initiator)：一个几何形状
    - 生成元(generator)：初始生成元的每部分用一个模型替代，多次迭代后生成分形图



Process Modeling- Fractal Geometry（cont.)

- 实例1：gasket（垫圈建模：三角形递归细分）
  ref:/Angle8ECode/02/gasket2.js




```
function triangle(a, b, c)
{
    positions.push(a, b, c);
}

function divideTriangle(a, b, c, count)
{

    // check for end of recursion

    if ( count === 0 ) {
        triangle(a, b, c);
    }
    else {

        //bisect the sides

        var ab = mix( a, b, 0.5 );
        var ac = mix( a, c, 0.5 );
        var bc = mix( b, c, 0.5 );

        --count;

        // three new triangles

        divideTriangle( a, ab, ac, count );
        divideTriangle( c, ac, bc, count );
        divideTriangle( b, bc, ab, count );
    }
}
```

Process Modeling- Fractal Geometry（cont.）

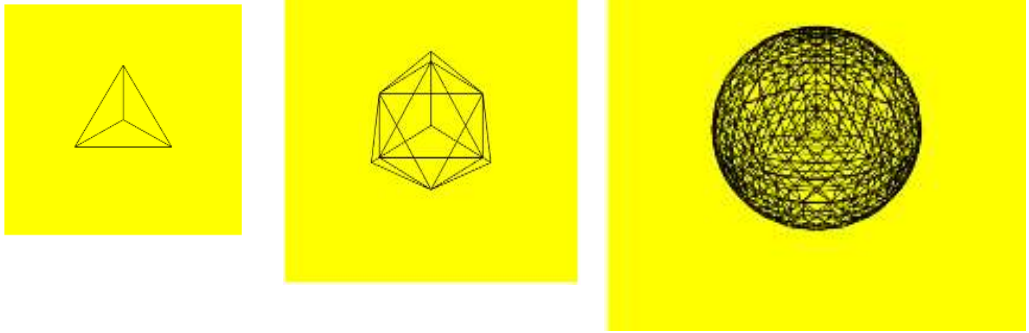
➢实例2：Wiresphere（四面体递归细分生成‘三角形球面网格’）

ref:/Angle8ECode/06/wireSphere.js

✓step1.初始为在单位四面体，有四个面，四个顶点在单位球面上。

✓Step2.取每条边上的中点P，将每个面划分为4个小三角形面，然后对新顶点进行归一化（即长度归约到1），也就是把新顶点“撑”到单位球面上。

✓step3.不断重复step2得到球面网格



```
function triangle(a, b, c) {
        pointsArray.push(a);
        pointsArray.push(b);
        pointsArray.push(c);
        index += 3;
}
function divideTriangle(a, b, c, count) {
    if ( count > 0 ) {

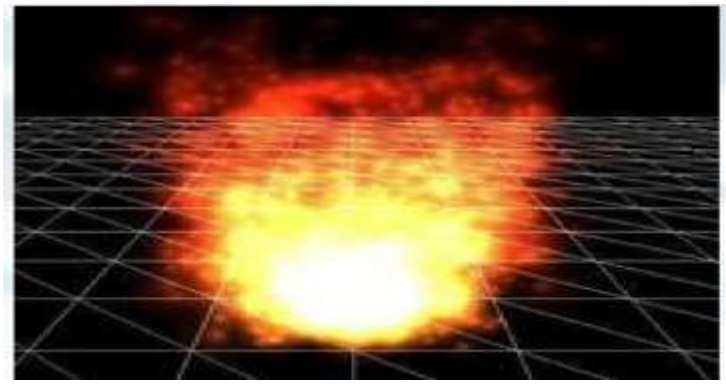
        var ab = normalize(mix( a, b, 0.5), true);
        var ac = normalize(mix( a, c, 0.5), true);
        var bc = normalize(mix( b, c, 0.5), true);

        divideTriangle( a, ab, ac, count - 1 );
        divideTriangle( ab, b, bc, count - 1 );
        divideTriangle( bc, c, ac, count - 1 );
        divideTriangle( ab, bc, ac, count - 1 );
    }
    else { // draw tetrahedron at end of recursion
        triangle( a, b, c );
    }
}
function tetrahedron(a, b, c, d, n) {
    divideTriangle(a, b, c, n);
    divideTriangle(d, c, b, n);
    divideTriangle(a, d, b, n);
    divideTriangle(a, c, d, n);
}
```

Process Modeling- Particle Systems

- particle systems微粒系统
  - 经常使用粒子系统模拟的现象有火、爆炸 、烟、水流、火花、落叶、云、雾、雪、尘、流星尾迹或者象发光轨迹这样的抽象视觉效果等等。

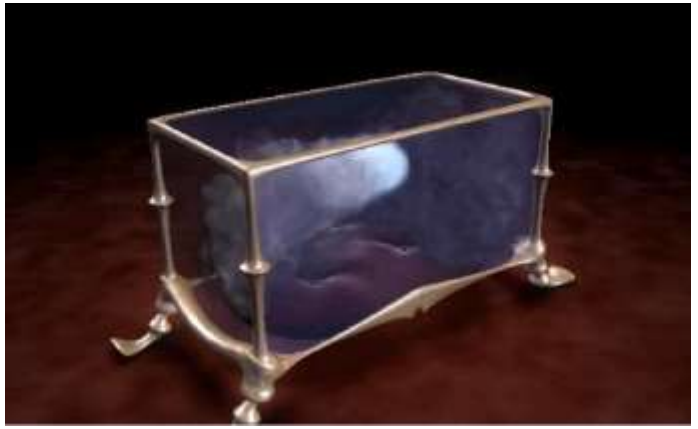




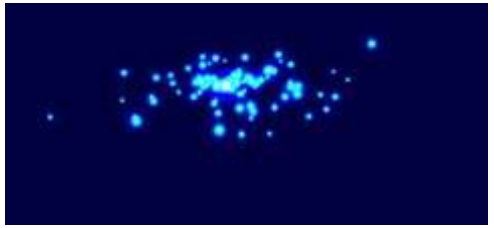


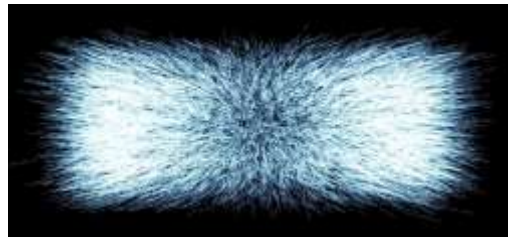
Process Modeling- Particle Systems(cont.)

- particle systems微粒系统(cont.)
  - 微粒系统是一组分散的微小物体集合，这些微小物体 的大小和形状可随时间变化，也就是说它们可以按照某种算法运动变化。
  - 尤其擅长描述随时间变化的物体，可以用于模拟自然景物或模拟其它非规则形状物体展示“流体”性质。
  - 微粒运动的模拟方式：随机过程模拟、运动路径模拟、力学模拟。



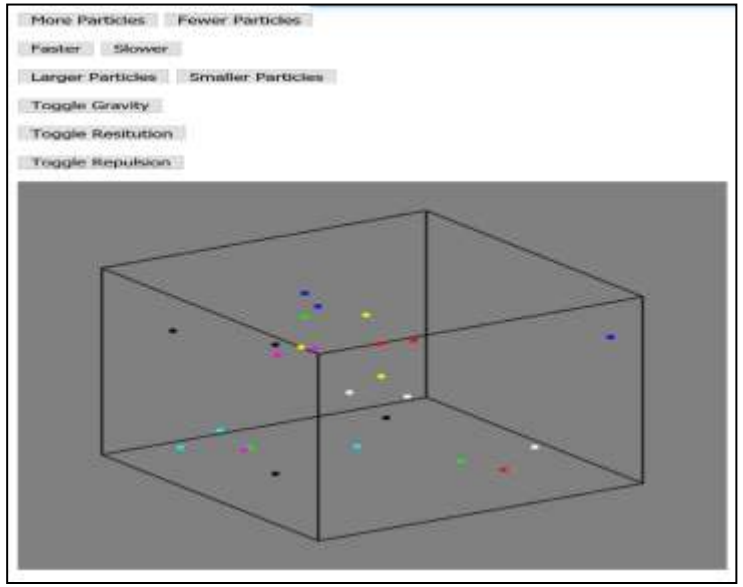






Process Modeling- Particle Systems(cont.)

- particle systems微粒系统(cont.)
  - 生成的两要素：粒子本身的造型 和 粒子的运动方式
  - 生成的两个过程： 模拟和渲染
    - 模拟粒子的碰撞，重力，粒子吸引力等物理运动规律
- **Ref: angle8ECode/10/particleSystem.html**



Summary

- 图形对象的构成(几何信息，非几何信息)

- 图形几何信息的数学表示(显式，隐式) 显式方便求值，隐式方便作判定

  1) 隐式:代数函数，CAD里常用实体造型技术(CSG)，水平集，距离场，分形，粒子系统等动态生成。

  如果是烟雾等非规则流体物体，使用SDF (Signed Distance Field(或 Signed Distance Function)符号距离函数表示。

  常采用ray marching 体渲染方法，SDF提供的距离信息告诉射线ray“安全”的前进步长，使得渲染复杂形态异常高效。

  2)显式:点云point cloud，网格polygon mesh，样条曲线曲面spline，三者递进，由小变大，可相互转换。

  渲染管线主要是triangle mesh的级别设计。

  如果是CAD,需要表示基本几何对象，并分析内部构造各部分关系，用”体素模型“。

  如果是游戏实时渲染，主要是“物体表面的表达“，用”边界模型或多边形网格“，然后转换为三角形网格后批量流水线进行渲染。

- 造型技术
  - Solid Modeling 实体造型*
    - B-reps(Boundary Representation)边界表示*
      - Mesh* 网格 (例子:立方体，墨西哥帽)
      - Spline Curve and Surface 样条曲线曲面*(连续性，拟合(插值/逼近))
  - Process Modeling 自然造型
    - Fractal geometry分形几何 (举例:球体生成)
    - Particle system粒子系统